

SZEGEDI TUDOMÁNYEGYETEM

Informatikai Intézet

SZAKDOLGOZAT

Épületgépészeti webshop kisebb cégek számára

Tóth Dávid László

PTNRYG

Gazdaságinformatika szakos

hallgató

Dr. Bilicki Vilmos, egyetemi docens

Szeged, 2026

Épületgépészeti webshop kisebb cégek számára
(TDLwebshop)

Mező	Tartalom
Készítette	Tóth Dávid László
Szak	Gazdaságinformatika szakos hallgató
Intézet / tanszék	Szegedi Tudományegyetem Informatikai Intézet
Témavezető	Dr. Bilicki Vilmos, egyetemi docens
Hely és év	Szeged, 2026

Feladatkiírás

A szakdolgozat célja egy épületgépészeti termékeket kezelő webshop és a hozzá kapcsolódó adminisztrációs felület megtervezése és megvalósítása. A rendszer kisebb vállalkozások számára nyújt digitális támogatást a termékek kezelésében, a készlet nyomon követésében és a vásárlói rendelések feldolgozásában. A vásárlói oldalon a felhasználók böngészhetnek és kereshetnek a termékek között, kosárba helyezhetik őket, majd rendelést adhatnak le számlázási adatokkal és átvételi időponttal. A kiszállítást a rendszer nem kezeli, a termékek átvétele személyesen történik az üzletben. Az adminisztrációs felületen a termékfeltöltés, a készletkezelés, a rendelésfeldolgozás és a vásárlói információk kezelése valósul meg, és a rendszer értesítést küld a készletcsökkenésről is. A rendelés véglegesítésekor a kiválasztott termékek levonódnak a készletből, így a vállalkozás pontosabban követheti a raktárállapotot.

Tartalmi összefoglaló

Téma megnevezése: TDLWebshop – épületgépészeti webshop és adminisztrációs rendszer.

A megadott feladat megfogalmazása: A feladat egy olyan webes alkalmazás létrehozása, amelyen kisebb épületgépészeti vállalkozások termékkatalógust kezelhetnek, vásárlói rendeléseket dolgozhatnak fel, helyszíni bolti vásárlásokat rögzíthetnek, és a vásárlókat egy katalógushoz kötött AI-asszisztens is segíti. A rendszer egyszerre kezeli a vásárlói oldalt és az adminisztrációs folyamatokat egyetlen közös adatmodellre építve.

Megoldási mód: A feladat megoldásához Angular[1] keretrendszert használtam TypeScript nyelvvel, valamint a Firebase[2] felhő alapú szolgáltatásait. Az AI-asszisztens nyelvi modell hívását egy Cloudflare Worker[4] proxy közvetíti az OpenRouter[5] szolgáltatáshoz.

Alkalmazott eszközök, módszerek: A fejlesztés Visual Studio Code-dal történt. A Firebase[2] szolgáltatásai közül az Authenticationt, a Firestore[3] adatbázist és a Hostingot használtam. A folyamatos integráció GitHub Actions[6] segítségével fut, így a build és a tesztek minden fontosabb commit után automatikusan lefutnak.

Elért eredmények: A webalkalmazást a Firebase[2] Hosting szolgáltatásával hostoltam, a forráskódot GitHubra töltöttem fel. A rendszer reszponzív megjelenítésű, így asztali gépen, laptopon, tableten és mobilon is használható. A vásárlói oldal mellett egy összetett admin felület is része a rendszernek, amely a termékfeltöltést, a rendeléskezelést, a készletfigyelést, a helyszíni vásárlást és a jogosultságkezelést is támogatja, valamint PDF-bizonylatot is képes generálni.

Kulcsszavak: webshop, épületgépészet, Angular[1], Firebase[2], AI-asszisztens, helyszíni vásárlás, jogosultságkezelés.

Tartalomjegyzék

Tartalom

Feladatkiírás	3
Tartalmi összefoglaló	4
Tartalomjegyzék	5
1. Bevezetés, problémafelvetés és célkitűzés	7
1.1. A megoldandó probléma	7
1.2. Célkitűzés és MVP-határ	7
2. Piaci és területi összehasonlítás	9
2.1. A saját rendszer értéke.....	10
3. Követelmények és use case-ek	10
3.1. Fő use case-ek.....	12
4. GUI/UX tervezés	14
5. Technológiai háttér	19
5.1. Alternatívák és döntések.....	20
6. Architektúra és adatáramlás.....	21
6.1. Modulok	22
7. Adatmodell	23
8. Megvalósítás kulcsfolyamatai	25
8.1. Checkout és rendelésmentés	25
8.2. Státusz, audit és készlet	25
8.3. Helyszíni vásárlás	25
8.4. CSV-import	26
8.5. PDF-bizonylat.....	26
8.6. AI-asszisztens	26
9. Biztonság és adatvédelem.....	27

9.1. MVP-korlátok biztonsági szempontból	29
10. Tesztelés	30
10.1. Build és CI	31
10.2. Kézi tesztjegyzőkönyv	31
11. Bizonyítékok összefoglalása	32
12. Reprodukálhatóság, CI és deploy	33
13. Mesterséges intelligencia használata a fejlesztés során	34
14. Összefoglalás, korlátok és továbbfejlesztés	36
Irodalomjegyzék	37
Mellékletek	37
Nyilatkozat	66

1. Bevezetés, problémafelvetés és célkitűzés

A TDLWebshop egy épületgépészeti webshop, amit kisebb szakkereskedéseknek készítettem. A vásárlói oldalt és az admin felületet egyetlen rendszerbe építettem össze, mert a téma választásakor az motivált, hogy van rálátásom ilyen boltok működésére. A legtöbb helyen a rendeléseket még telefonon vagy kézzel kezelik, a készletet pedig külön táblázatokból követik. Emiatt egy webshop akkor lesz tényleg hasznos, ha a vásárlói felületen túl a háttérfolyamatokra is választ ad. A dolgozatban bemutatott rendszer ezeket próbálja egy közös adatmodellre építve összeérni: termékek, kosár, checkout, rendeléskövetés, admin funkciók, jogosultságkezelés és biztonsági szabályok. Az MVP-t úgy alakítottam ki, hogy hallgatói méretben is teljes legyen, és más környezetben is reprodukálható maradjon: a repóban benne van a futtatási leírás, a környezeti változók mintája, a CI és a tesztelési anyag.

1.1. A megoldandó probléma

Az épületgépészeti termékek választéka összetett és sokrétű. Egy radiátor, csaptelep, szivattyú vagy szellőztető kiválasztásánál nem csak az ár számít, hanem a méret, a teljesítmény, az alkalmazási terület és a kompatibilitás is. A vásárlónak ezért nem elég csak felsorolni a termékeket, az oldálnak segítenie kell a választásban. Az admin oldalon más típusú probléma van: egy kisebb kereskedésnél nem csak webes rendelések érkeznek, hanem helyszíni vásárlások, mentett vevők, készletmozgások és bizonylatok is. Ha ezeket külön rendszerek kezelik, könnyen lesznek belőle hibák és eltérések. A TDLWebshop ezért egy közös adatmodellre építve fogja össze a két oldalt, így a termék- és rendelésadatok mindkét nézetben ugyanúgy jelennek meg.

1.2. Célkitűzés és MVP-határ

A projekt célja nem egy ipari webshoprendszer lemásolása volt, hanem egy működő MVP, amely a legfontosabb vásárlói és admin folyamatokat lefedi. Ez tudatos döntés volt, mert egy szakdolgozatban könnyen túl sok félkész funkció gyűlhet össze, ami gyengíti az összképet. Az alábbi táblázat foglalja össze, mi tartozik az MVP-be és mi kerül a továbbfejlesztések közé.

1. táblázat – Az MVP-határ összefoglalása

MVP része	Rövid indoklás	Állapot
Terméklista és termékadatlap	A webshop alapja, a vásárlói tájékozódás kiindulópontja.	Megvalósítva
Kosár és checkout	A vásárlói út legfontosabb üzleti folyamata.	Megvalósítva
Rendeléskövetés profilból	Bizalmat és áttekinthetőséget ad a vásárlónak.	Megvalósítva
Admin termék- és készletkezelés	A bolt működtetéséhez szükséges belső folyamat.	Megvalósítva
4.5. ábra. Helyszíni vásárlás – mentett vásárló és tételek rögzítése.	Domainhez illeszkedő B2B/üzleti különbség.	Megvalósítva
PDF-bizonylat/számla	Adminisztrációs bizonyíték és szakdolgozati érték.	Megvalósítva
AI-asszisztens	Katalógushoz kötött, korlátozott szakmai segéd.	Prototípus
Bankkártya-elfogadás	Valós pénzügyi integráció magasabb kockázattal.	Továbbfejlesztés
ERP-integráció	Vállalati szintű illesztés külső rendszerekkel.	Továbbfejlesztés

2. Piaci és területi összehasonlítás

A TDLWebshop pozicionálásához érdemes átnézni, milyen funkciókat kínál egy átlagos webshop, és miben más egy épületgépészeti rendszer. Nem azt akarom megmutatni, hogy a TDLWebshop mindenkit felülmúl, hanem azt, hogy a választott funkciók valódi igényekre épülnek. Az általános webshopoknál a vásárlói oldal jellemzően erős — van terméklista, kategóriák, kereső, kosár —, de az adminisztráció vagy a háttérben marad, vagy egy külön vállalati rendszerben fut. A TDLWebshop ebben más, mert a vásárlói felület mellett az admin működés és a helyszíni vásárlás is része a rendszernek. Az épületgépészeti területen a vásárló sokszor szakmai döntéshez vár támogatást, ezért hasznos lehet a kategóriák közötti eligazodás, a készletinformáció és a katalógushoz kötött AI-asszisztens — utóbbi annyiban, amennyire a korlátai egyértelműek. A magyar piacon több hasonló profilú szereplő működik – például a Megatherm (országos hálózat), a Ferenczi Épületgépészet (Budapest környéke), a Barta Thermoker (széles katalógus), a Hegedűs Kft. (Szolnok) vagy a Gépész Holding –, ezek többnyire a hagyományos B2C-webshop modellt követik, az admin működést pedig külön belső rendszerben kezelik.

2. táblázat – Piaci funkciók összehasonlítása

Szempon	Általános webshop	Épületgépészeti webshop	TDLWebshop megoldása
Terméklista	Általános kategóriák és keresési lehetőség.	Szakmai kategóriák, méretek, készlet.	Fűtés, víz, hűtés, szellőzés, szerelési anyagok, lakossági megoldások.
Admin folyamat	Többnyire nem látható a felhasználónak.	Készlet és rendelés gyors kezelése fontos.	Admin felület, CSV-import, készletfigyelés, helyszíni vásárlás.
Rendeléskövetés	Alap státuszok.	Fontos a készlet és teljesítés követése.	Profilból látható rendelések és admin státuszváltás.

B2B jelleg	Gyakran külön rendszer.	Céges kedvezmény és mentett vásárló fontos.	Mentett vásárlók, céges jelölés, helyszíni rögzítés.
AI-támogatás	Ritkán katalógushoz kötött.	Szakmai kérdéseknél hasznos lehet.	Katalógusalapú, korlátozott AI-asszisztens OpenRouter-proxyn keresztül.

2.1. A saját rendszer értéke

A TDLWebshop értéke nem egyetlen funkcióban rejlik, hanem abban, hogy a folyamatok összeérnek. A vásárló terméket keres, kosárba teszi, leadja a rendelést, majd a profiljából követi az állapotát. Az admin ugyanezekon az adatokon dolgozik tovább: módosítja a rendelést, figyeli a készletet, PDF-bizonylatot készít vagy helyszíni vásárlást rögzít. Szakdolgozati szempontból a projekt azért is érdekes, mert több technikai terület kapcsolódik benne össze. A kliensoldal Angular-alapú, az adattárolás és a hitelesítés Firebase-en megy keresztül, a biztonságot Firestore-szabályok adják, az AI-asszisztens pedig egy Cloudflare Worker-proxyn keresztül éri el a nyelvi modellt. A CI-ellenőrzés ezeket a részeket egyben tartja, és így a projekt nem csak egy webshop felület, hanem egy összekapcsolt, működő rendszer. A vizsgált magyar épületgépészeti webshopok többségénél nem jelenik meg ilyen szinten integrálva a vásárlói felület, a helyszíni bolti vásárlás és a katalógushoz kötött AI-asszisztens egyetlen közös Firestore-adatmodellre építve – ez a kombináció a TDLWebshop önálló mérnöki értéke.

3. Követelmények és use case-ek

A követelmények rögzítése segít abban, hogy ne csak felsoroljam, mit tud a rendszer, hanem azt is megmutassam, milyen igényekhez kapcsolódnak az egyes funkciók, és hogyan ellenőrizhető a működésük. A TDLWebshop esetében három követelménycsoportot különítettem el. A vásárlói oldalon a termékkeresés, a kosár, a checkout, a profil és a rendeléskövetés tartozik ide. Az adminisztrációhoz a termékfeltöltés, a rendelés- és készletkezelés, a vásárlókezelés, a kuponok és a PDF-generálás. A harmadik csoportba a technikai és minőségi elvárások kerültek:

jogosultságok, biztonság, tesztelhetőség és reprodukálhatóság. Az alábbi traceability-táblázat azért hasznos, mert minden követelményt összeköt egy use case-szel, a megvalósító modullal és a működés bizonyítékával.

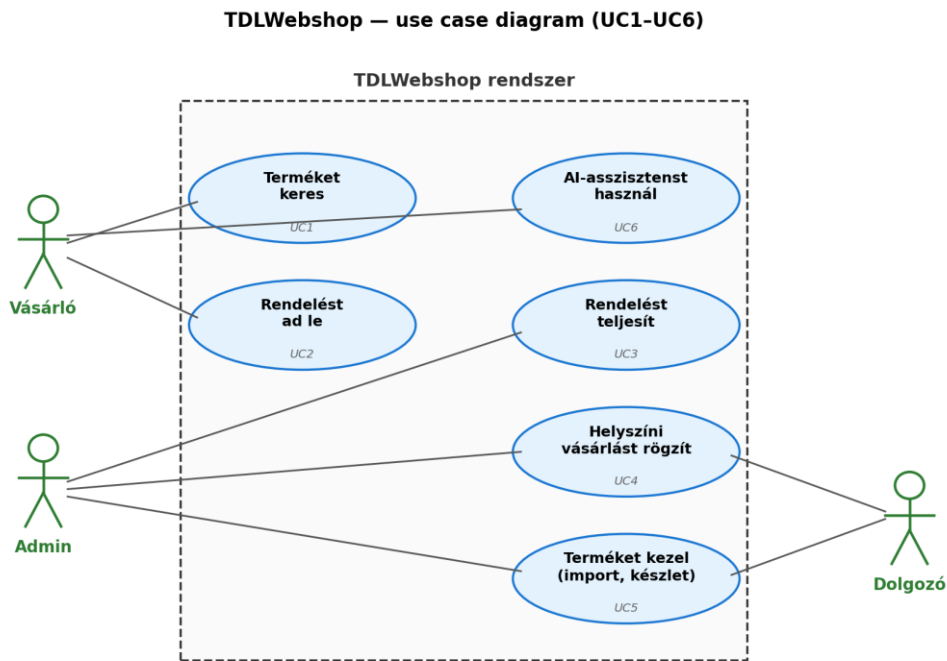
3. táblázat – Követelmény-traceability összefoglaló

Azonosító	Követelmény	Use case	Modul / bizonyíték
K1	A felhasználó tudjon terméket keresni és kategóriák szerint böngészni.	Termékek böngészése	Terméklista oldal, keresési mező, kategóriák
K2	A felhasználó tudjon terméket kosárba tenni és mennyiséget módosítani.	Kosárkezelés	CartService, kosár oldal
K3	A checkout ellenőrizze az e-mailt, telefonszámot és kötelező adatokat.	Rendelés leadása	checkout.ts validációs logika
K4	Admin rendeléskezelés és teljesítés: állapot módosítása, auditbejegyzés, készletváltozás.	Admin rendeléskezelés	admin.ts, order.service.ts
K6	A dolgozó csak korlátozott admin funkciókat érjen el.	Dolgozói felület	Firestore rules, admin jogosultsági logika
K7	A PDF-bizonylat tartalmazza a rendelést, vevőt, tételeket és összegeket.	Számla / bizonylat letöltése	invoice.service.ts
K8	Az AI-asszisztens ne találjon ki terméket, csak katalógushoz kötötten ajánljon.	AI-kérdés megválaszolása	chatbot-llm.service.ts, Worker-proxy

3.1. Fő use case-ek

4. táblázat – Use case-ek rövid összefoglalása

Use case	Név	Leírás
UC1	Vásárló terméket keres és kosárba teszi	A vásárló kategóriát választ vagy keresőszót ad meg, megnyitja a terméket, majd kosárba helyezi.
UC2	Vásárló rendelést ad le	A vásárló kitölti a checkout-űrlapot, a rendszer validál, majd létrehozza a rendelést.
UC3	Admin teljesíti a rendelést	Az admin megnyitja a rendelést, ellenőrzi az adatokat, státuszt módosít, a rendszer auditot és készletváltozást rögzít.
UC4	Admin helyszíni vásárlást rögzít	Az admin mentett vásárlót választ vagy új adatot ad meg, tételt ad a vásárláshoz, majd PDF-bizonylatot generál.
UC5	Dolgozó terméket kezel	A dolgozó a számára engedélyezett admin funkciókkal terméket tölthet fel és készletet ellenőrizhet.
UC6	Felhasználó AI-asszisztenst használ	A felhasználó katalógushoz vagy épületgépészethez kapcsolódó kérdést tesz fel, a rendszer korlátozott választ ad.

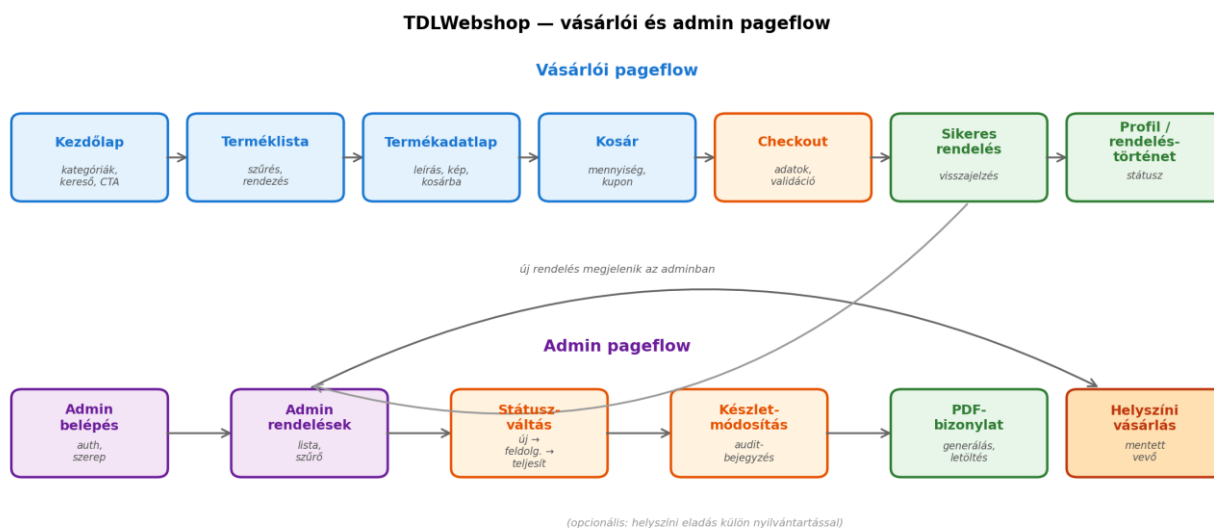


2. ábra. TDLWebshop use case diagram (UC1–UC6).

A diagramon három aktort különböztetek meg, és nem véletlenül. A Vásárló kívülről érkezik: regisztráció nélkül is böngészheti a katalógust, de rendeléshez már bejelentkezés kell. Az Admin teljes körű hozzáféréssel rendelkezik – termékeket vesz fel, rendeléseket módosít, helyszíni vásárlást rögzít, és a többi felhasználó szerepkörét is állítja. A Dolgozó egy köztes szerepkör, amit azért építettem be, mert egy boltban nem minden alkalmazottnak kell mindenhez hozzáférnie: a dolgozó rögzíthet helyszíni vásárlást és kezelheti a terméket, de nem módosíthatja a jogosultságokat vagy az alapvető rendszerbeállításokat. A szerepkör-eltávolítás azért fontos, mert a jogosultságkezelés nem csak a UI elrejtésével működik – a Firestore-szabályok szerveroldalon érvényesítik, ki mit láthat és módosíthat.

4. GUI/UX tervezés

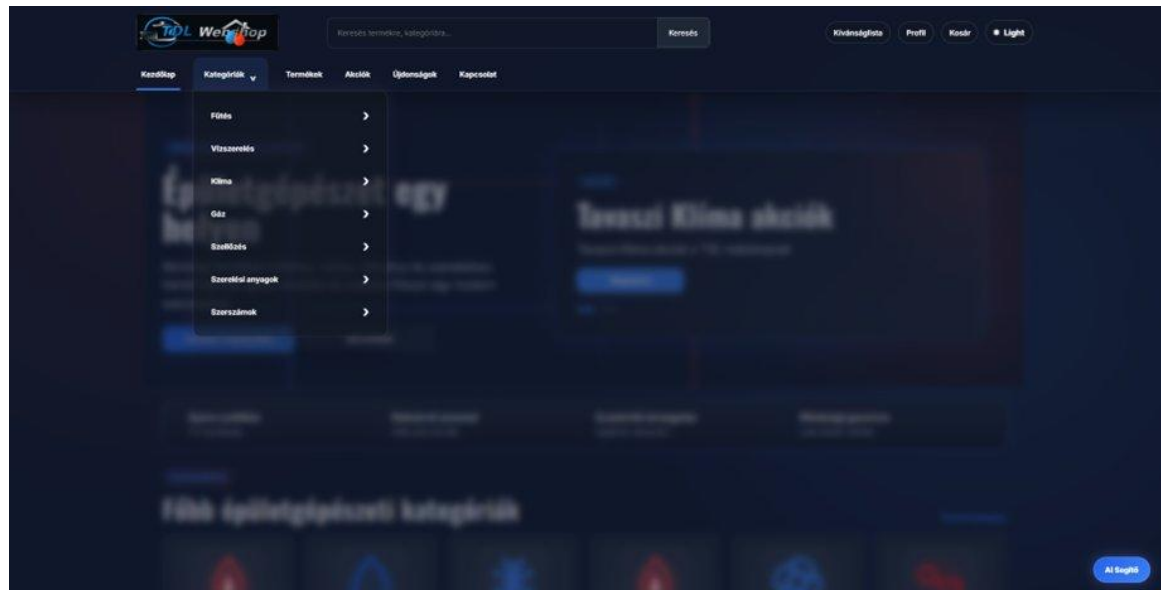
A felület kialakításánál modern, sötét alapú, ipari hangulatú arculatot szerettem volna. Ez illik az épületgépészeti témához, és a TDLWebshop logójával egységes vizuális világot ad. A dark mode az alapértelmezett megjelenés, a light mode csak kényelmi alternatíva. A layout nem véletlenül néz ki úgy, ahogy: a keresősáv, a kategóriák, a kosár és a profil olyan helyre került, hogy a felhasználó gyorsan eljusson a termékig és a rendelésig. Az admin felület ezzel szemben sűrűbb és táblázatos, mert ott a gyors kezelés számít, nem a látvány. A részletes UX-dokumentáció (pageflow, screens-tábla, journeys, design system) a repó docs/ux/ mappájában található, ezért ez a fejezet csak a legfontosabb felhasználói felületeket mutatja be.



3. ábra. TDLWebshop pageflow – vásárlói és admin folyamatok.

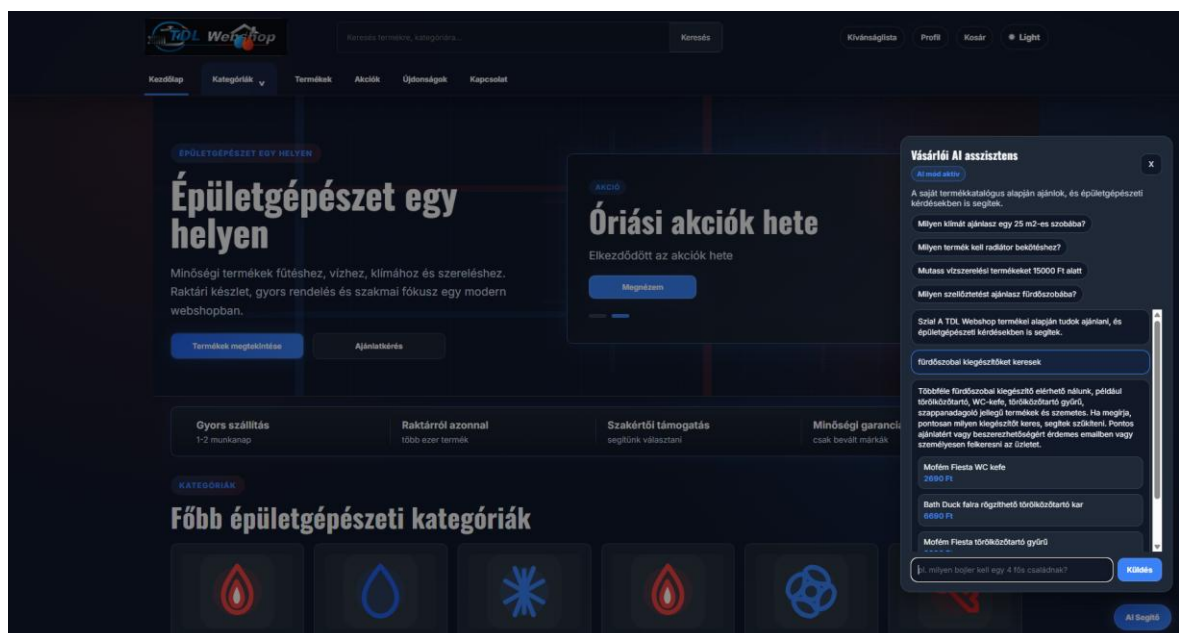
A pageflow két párhuzamos folyamatot mutat. A felső sávban a vásárlói út lépései szerepelnek: a kezdőlapról indulva a kategóriák, a szűrt terméklista, a részletes termékadatlap, a kosár és a checkout vezet a sikeres rendelésig, majd a profil oldalon követhető a státusz. A checkoutnál narancssárga kiemelés jelzi, hogy ez a kritikus validációs pont, ahol megakadhat a folyamat (UC2). Az alsó sávban az admin folyamat látható: bejelentkezés, rendelések listája, státuszváltás (új → feldolgozás alatt → teljesítve), készletmódosítás és PDF-bizonylat-generálás (UC3, UC5). A két folyamat között kulcsfontosságú átkötés a középső szürke nyíl: amint a vásárlói oldalon létrejön egy rendelés, az ugyanazon a Firestore-dokumentumon keresztül megjelenik az admin felületen is – nincs külön rendelési modul, az adat egyetlen forrásból táplálkozik. A helyszíni vásárlás

oldalsó folyamatként jelenik meg, mert ez a webes vásárlással párhuzamos opcionális admin-funkció (UC4).



4.1. ábra. A TDLWebshop kezdőlapja dark módban, kategóriák legördülő menüvel.

A kép a felület alapdízajnját mutatja: a sötét háttér és a karakteres logó az épületgépészeti témához illő ipari arculatot követi, ezt tudatos tervezési döntésnek szántam. A felül elérhető keresősáv, a kategóriák legördülő menüje, valamint a kosár- és profil-ikonok minden oldalon stabilan jelen vannak; a felhasználó innen jut el a katalógusba (UC1: termékkeresés).



4.2. ábra. AI-asszisztens felület katalógushoz kötött kérdés-válasszal.

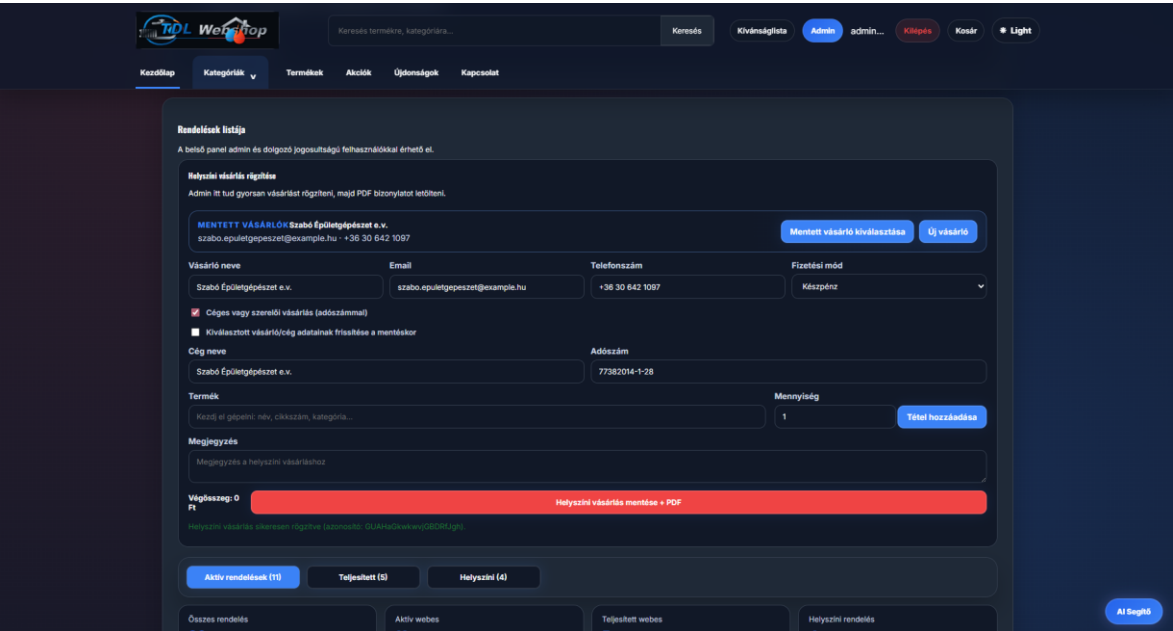
Az AI-asszisztens egy lebegő ablakban jelenik meg, amelyben a beszélgetés a katalógusból tájékozódó nyelvi modellel folyik (UC6). A válasz a webshopban szereplő termékekre hivatkozik vissza; ha a kérdés ettől távoli vagy nincs releváns találat, az asszisztens jelzi a korlátját, és nem ad konkrét ajánlást.

4.3. ábra. Checkout oldal validációs hibaüzenettel hibás adat esetén.

A checkoutnál a kép egy konkrét hibás bemenetet mutat (rosszul formázott e-mail és rövid telefonszám). A rendszer ezeket nem engedi tovább: piros kiemeléssel jelzi a javítandó mezőt, és nem véglegesíti a rendelést. Ez egyrészt a 10.2 negatív tesztet vizuális bizonyítéka, másrészt a UC2 (rendelésleadás) kritikus validációs pontját demonstrálja.

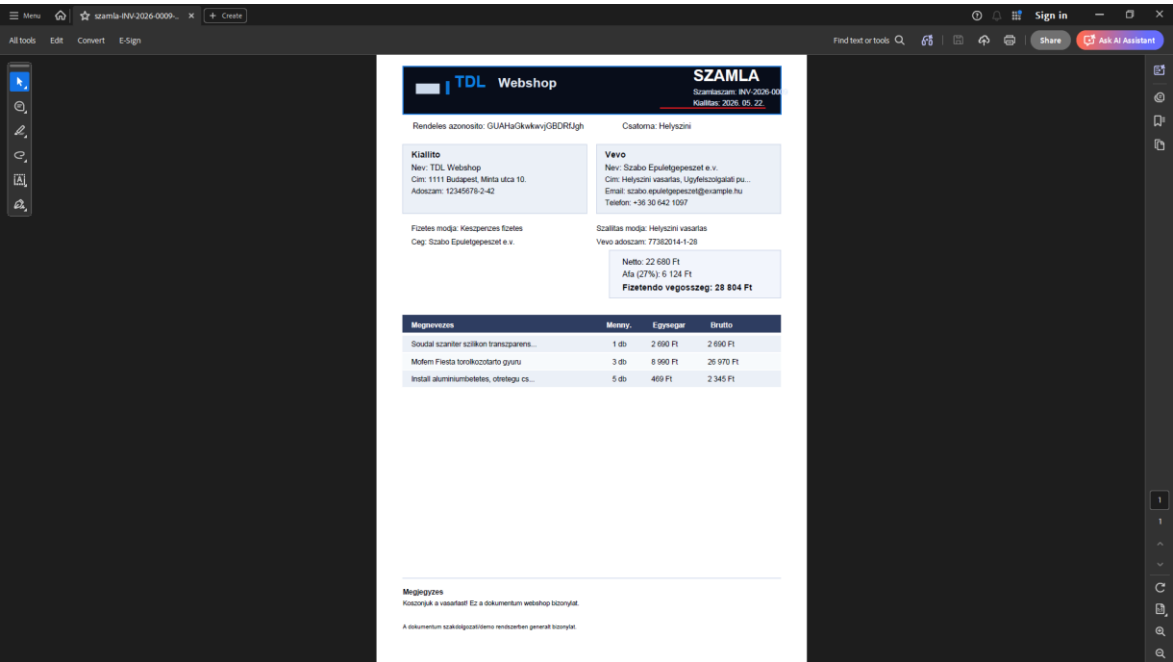
4.4. ábra. Admin felület rendelések áttekintésével és státuszváltással.

Az admin rendeléslista (UC3) státusz szerinti szűrést, sorrendezést és tételenkénti megnyitást tesz lehetővé. A státuszváltás itt érvényesíti a 8.2 alfejezetben leírt tranzakciós logikát: a rendelés állapota, a kapcsolódó auditbejegyzés és a készletmódosítás egyszerre, atomi módon történik.



4.5. ábra. Helyszíni vásárlás – mentett vásárló és tételek rögzítése.

A helyszíni vásárlás felület (UC4) a domain szempontjából egyedi funkcionalitás: egy boltban előforduló offline vásárlás kezelését támogatja. A mentett vevő kiválasztása csökkenti a kézzel begépett adatok mennyiségét, a tétel-rögzítés pedig a webes kosárral lényegében azonos UI-t használ – ezzel a két csatorna konzisztens marad.



4.6. ábra. PDF-bizonylat rendelési adatokkal, tételekkel és végösszeggel.

A generált PDF-bizonylat tartalmazza a kiállító és a vevő adatait, a rendelésazonosítót, a tételsorokat, valamint a nettó és bruttó összegeket. A layoutot úgy alakítottam ki, hogy több tételes vásárlás esetén is olvasható maradjon és ne csússzanak össze a blokkok – ezt a megoldást a 8.5 alfejezet részletezi (UC3 zárólépése).

5. Technológiai háttér

A TDLWebshop Angular-alapú egyoldalas webalkalmazás. Az Angular mellett azért döntöttem, mert a komponensalapú felépítés egy nagyobb felületnél átláthatóbb kódot eredményez: a terméklistát, a kosarat, a checkoutot, a profilt és az admin felületet külön részekre tudtam bontani. Az adattárolást és a hitelesítést Firebase szolgáltatások végzik. A Firestore dokumentumalapú modellje jól illik a projekt adataihoz, a Firebase Authentication pedig kényelmes megoldást ad a szerepkörökhöz kötött belépésre. Az AI-asszisztens külső nyelvi modellt használ, de a kliens nem közvetlenül szólítja meg az OpenRouter API-t. Helyette egy Cloudflare Worker-proxy közvetít, így az API-kulcs nem kerül a frontend kódjába.

5. táblázat – A rendszerben használt technológiák

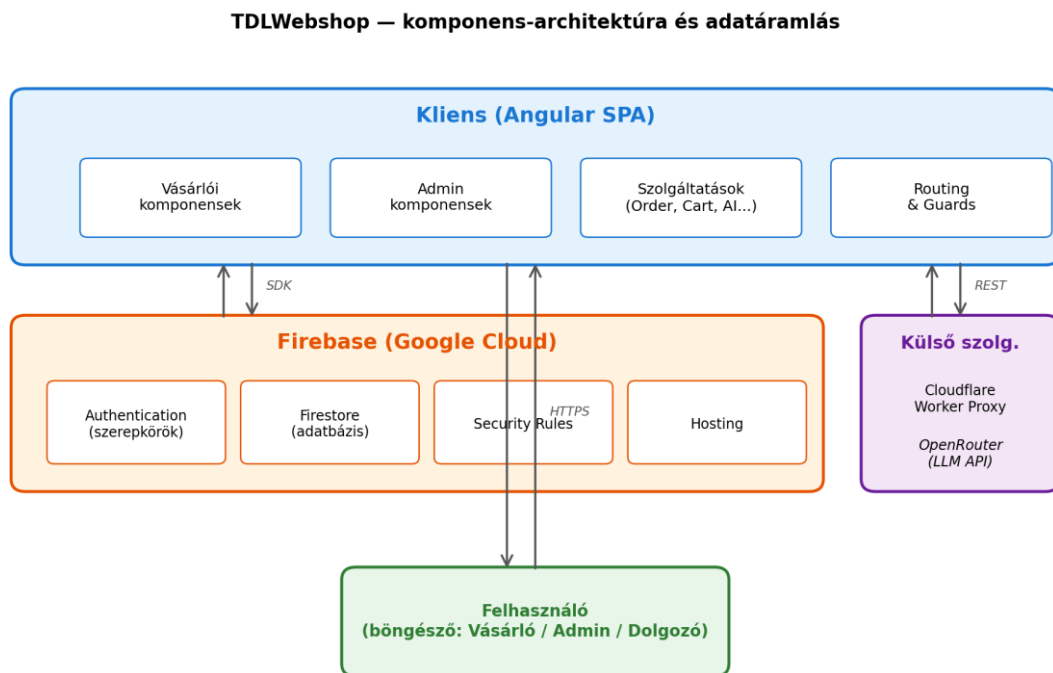
Technológia	Szerep a rendszerben	Indoklás
Angular	Frontend-keretrendszer	Komponensalapú, TypeScript-alapú, jól használható nagyobb felülethez.
TypeScript	Alkalmazáslogika	Átláthatóbbá teszi a komponenseket és a szolgáltatásokat.
Firebase Authentication	Felhasználóazonosítás	E-mailes belépés és szerepkörhöz kötött hozzáférés.
Cloud Firestore	Adattárolás	Dokumentumalapú modell és szabályalapú biztonsági réteg egyszerre.
Cloudflare Worker	AI-proxy	API-kulcs védelme és külső hívások kezelése.
GitHub Actions	CI-ellenőrzés	Build és tesztfuttatás automatikus ellenőrzése.

5.1. Alternatívák és döntések

A backendet megírhattam volna saját Node.js/Express szerverrel is, de végül a Firebase mellett döntöttem. Az hitelesítés, az adatbázis, a hosting és a biztonsági szabályok így egy helyen kezelhetők, és nem kellett külön szerveroldalt fenntartani. A bankkártyás fizetés és a pénzügyi integráció szándékosan maradt ki: egy hallgatói projektben technikailag és jogilag is nagy kockázatot jelent, ezért a fizetési módok csak adminisztratív szinten jelennek meg. Az AI-asszisztensnél is fontos volt a határok megszabása: nem szakmai tanácsadó, hanem a katalógusból tájékozódó segédfunkció. Ha a kérdés ettől távoli, vagy nincs megfelelő terméktalálat, akkor inkább nem ad konkrét választ.

6. Architektúra és adatáramlás

A rendszer három rétegből áll. A klienst az Angular-alkalmazás adja, ide tartozik a felhasználói felület és a komponensek. A Firebase-réteg felel a hitelesítésért, a Firestore-adatbázisért, a biztonsági szabályokért és a hostingért. Külső szolgáltatásként pedig az AI-proxy és az OpenRouter működik együtt a rendszerrel. Az Angular oldalán a komponensek a megjelenítést és az interakciókat kezelik, az adatműveleteket pedig a szolgáltatások végzik (kosár, rendelés, számla, profil, AI-asszisztens, termékkezelés). Az adatáramlás a checkout-folyamaton látszik a legjobban: a vásárló megadja az adatait, a komponens ellenőrzi azokat, majd a rendelési szolgáltatás létrehoz egy Firestore-dokumentumot. Ezt később az admin felület olvassa ki és kezeli tovább; státuszt módosít, auditbejegyzést készít, és szükség szerint frissíti a készletet.



1. ábra. TDLWebshop komponens-architektúra és adatáramlás.

Az ábrán látható felosztás azért így alakult ki, mert mindhárom rétegnek más a feladata, és külön is fejleszthető vagy cserélhető. A kliens (Angular SPA) csak a megjelenítésért és a felhasználói interakcióért felel; minden adatműveletet a szolgáltatásokon keresztül indít, és sosem hív közvetlenül külső API-t. A Firebase ezért került középre: így nem kellett saját szerveroldali alkalmazást írnom, viszont a hitelesítés, a Firestore-adatbázis és a biztonsági szabályok egy ökoszisztémában maradnak. Az AI-asszisztens szándékosan nem közvetlenül szól az OpenRouter API-hoz, hanem egy Cloudflare Worker-proxyn keresztül. Ennek az az

oka, hogy az API-kulcs nem kerülhet a frontend kódba (mert a böngésző fejlesztői konzoljából bárki kiolvashatná); a Worker környezeti változóban tárolja a kulcsot, és csak a saját TDLWebshop domainről érkező kéréseket szolgálja ki.

6.1. Modulok

6. táblázat – Fő modulok és kapcsolataik

Modul	Feladat	Kapcsolódó adat
Home / Product UI	Kezdőlap, kategóriák, termékkartyák.	products
Cart	Kosár állapot, mennyiség, összegzés.	local state, products
Checkout	Adatbekérés, validáció, rendelés létrehozása.	orders
Profile	Felhasználói adatok és rendelések.	users, orders
Admin	Termék, rendelés, készlet és vásárlókezelés.	products, orders, savedCustomers
Invoice	PDF-bizonylat generálása.	orders, order items
AI-asszisztens	Katalógushoz kötött válaszadás.	products, Worker-proxy

A teljes implementáció az M2 mellékletben szerepel (workers/openrouter-proxy/src/index.js – AI proxy logika).

7. Adatmodell

Az adatkezelés Firestore-ra épül, így az adatok nem relációs táblákban tárolódnak, hanem gyűjteményekben és dokumentumokban. Ez gyorsabb fejlesztést tesz lehetővé, viszont az entitások szerkezetét érdemes előre átgondolni. A főbb gyűjtemények a termékekhez, a rendelésekhez, a felhasználókhoz és a mentett vásárlókhoz kapcsolódnak, illetve a kuponokhoz, a számlákhoz és az auditbejegyzésekhez. Az volt a fontos szempont, hogy a vásárlói és az admin felület közös adatbázisra épüljön: a vásárló listában és részletes nézetben látja a termékeket, az admin pedig szerkeszti, importálja és készletszinten kezeli. A rendelések ugyanígy érhetők el két oldalról; a vásárló a profilból követheti őket, az admin pedig státuszváltást és teljesítést tud végezni rajtuk. A dokumentumalapú megoldás akkor marad stabil, ha a Firestore-szabályok és az alkalmazásoldali validációk együtt dolgoznak.

Miért dokumentum-alapú adatbázist választottam, nem klasszikus relációt? Egy webshop adatai (termékek képekkel és attribútumokkal, rendelések beágyazott tételsorokkal, vásárlói profilok rendeléstörténettel) természetesen illeszkednek egy dokumentum-alapú modellre, és így nem kellett külön táblákat és JOIN-okat fenntartanom egy-egy logikai entitáshoz. A Firestore emellett valós idejű frissítéseket is támogat: ha az admin felületen módosul egy rendelés státusza, a vásárlói profil oldal akár újratöltés nélkül is láthatja. A trade-off, hogy az aggregált lekérdezések (pl. „összes termék kategóriánként”) nehezebbek, mint SQL-ben – ezeket a kliens oldalon, indexelt mezőkkel oldottam meg, ami egy ekkora rendszernél még jól skálázik.

A főbb gyűjtemények közötti kapcsolatot érdemes konkrétan is végignézni. A products gyűjtemény a termékek adatait tárolja, és kétféle folyamat is módosíthatja: az admin oldalon a manuális termékfeltöltés és a CSV-import (admin/Termékkezelés), valamint a státuszváltáshoz kötött készlet-módosítás (OrderService). Az orders gyűjtemény dokumentumai egy-egy rendelést írnak le, a beágyazott tételsorokkal együtt; a vásárlói oldalon a checkout hozza létre őket, az admin oldalon pedig státuszt és fizetést tudunk hozzá állítani. A users gyűjtemény a felhasználói profilt és a szerepkört tárolja, ezen alapulnak a Firestore-szabályok jogosultság-ellenőrzései. A savedCustomers a helyszíni vásárláshoz tartozik: itt mentjük a visszatérő bolti vevők adatait, hogy a következő vásárlás gyorsabb legyen, és nem kerülnek össze a regisztrált felhasználókkal. A coupons gyűjtemény a kedvezménykódokat tárolja, a checkout során innen kerülnek érvényesítésre. Az audit gyűjtemény végül minden fontos admin-műveletet (státuszváltás, készletmódosítás,

jogosultság-változás) rögzít – ez nem csak biztonsági visszakövethetőség, hanem egy egyszerű módja annak, hogy lássuk, ki mit változtatott, és mikor. Azért szétbontva tárolom őket, mert a Firestore biztonsági szabályok gyűjtemény-szinten működnek, és így pontosabb jogosultsági réteget tudtam felépíteni rájuk.

7. táblázat – Adatmodell fő entitásai

products	name, sku, category, price, stock, images, discount	Terméklista, admin termékkezelés, AI-katalógus
orders	customer, items, total, status, paymentMethod, createdAt	Checkout és admin rendeléskezelés
orderStatusAudit	orderId, oldStatus, newStatus, actor, createdAt	Státuszváltások nyomon követése
users	email, role, disabled, profileData	Jogosultságok és profiladatok kezelése
savedCustomers	name, email, phone, company, taxNumber, disabled	Helyszíni vásárlás és mentett vásárlók kezelése
coupons	code, discount, active, validity	Kedvezménylogika kezelése
newsletter	email, createdAt	Hírlevél- feliratkozások tárolása

A teljes kódrészlet az M2 mellékletben található (order.service.ts 269–302: helyszíni vásárlás mentése).

8. Megvalósítás kulcsfolyamatai

Ez a fejezet a rendszer működésének gyakorlati oldalát mutatja be. Sorra veszem a checkoutot, a rendelési státuszok kezelését, a helyszíni vásárlást, a CSV-importot, a PDF-bizonylat generálását, a jogosultságkezelést és az AI-asszisztens működését. A fejezet súlypontja nem a felhasznált technológiákon, hanem azon van, hogy a részek a gyakorlatban hogyan kapcsolódnak össze egy működő rendszerre.

8.1. Checkout és rendelésmentés

A checkout a vásárlói folyamat egyik legfontosabb pontja, mert itt dől el, hogy a böngészésből és a kosárkezelésből valóban létrejön-e egy rendelés. A felhasználó itt adja meg a számlázási és kapcsolattartási adatokat, a rendszer pedig ellenőrzi többek között az e-mail-címet és a telefonszámot. A felhasználói adatok, a kosár tartalma, a fizetési mód és az esetleges kedvezmény egy közös rendelési objektumba kerülnek. Ha a mentés sikeres, a vásárló visszajelzést kap, az admin felületen pedig azonnal megjelenik az új rendelés. MVP-szintű korlát, hogy bizonyos ár- és kedvezményadatok kliensoldalról érkeznek; éles környezetben szerveroldali újraszámolásra is szükség lenne, ezt a projekt korlátai között érdemes feltüntetni.

8.2. Státusz, audit és készlet

A rendelések teljesítése vagy törlése nem csak a rendelés állapotát érinti, hanem a készletet is, ezért a státuszváltás, az auditbejegyzés és a készletmódosítás összhangban történik. A státuszváltás tranzakciós logikán keresztül fut, így egy helyen kezelhető a rendelés frissítése, az auditnyom létrehozása és a készlet módosítása. Ha a folyamat közben hiba történik, a rendszer visszatér az előző állapothoz, és a háttérfolyamatok következetesen kapcsolódnak a felhasználói műveletekhez.

8.3. Helyszíni vásárlás

A helyszíni vásárlás funkció jól illeszkedik a projekt témájához, mert egy épületgépészeti boltban nem minden értékesítés történik weben keresztül. Az admin vagy a dolgozó mentett vásárlót választhat, új adatokat adhat meg, terméket kereshet, mennyiséget állíthat be, majd a vásárlás után bizonylatot generálhat. Ez a folyamat azért értékes, mert nem csak egy általános webshopos működést mutat meg, hanem egy belső bolti helyzetet is modellez. A mentett vásárlói adatok újra felhasználhatók, így a visszatérő vevők kezelése gyorsabb és egyszerűbb.

8.4. CSV-import

A termékfeltöltést a kézi rögzítés mellett CSV-import is támogatja, ami akkor különösen hasznos, ha egyszerre nagyobb mennyiségű terméket kell felvinni árakkal, kategóriákkal, képekkel és SKU-val együtt. Az importálásnál különösen fontos a validáció, mert egy hibás oszlop vagy rossz elválasztás könnyen több hibás dokumentumot eredményezhetne, ezért a rendszer csak megfelelő szerkezetű adatok esetén engedi a mentést.

8.5. PDF-bizonylat

A PDF-bizonylat generálása az adminisztráció egyik látványos eredménye. A dokumentum tartalmazza a kiállító és a vevő adatait, a rendelés azonosítóját, a tételsorokat, a nettó és bruttó összegeket, valamint a fizetési és átvételi adatokat. Ez a funkció jól mutatja, hogy a rendszer nem csak adatbázis-rekordokat kezel, hanem strukturált dokumentumot is képes előállítani. A bizonylat elrendezésénél figyelni kellett arra, hogy a különböző blokkok ne csússzanak össze, és a dokumentum több tételes vásárlás esetén is olvasható maradjon.

8.6. AI-asszisztens

Az AI-asszisztens célja az, hogy a saját termékkatalógusból kiindulva segítse a felhasználót. A válaszok ezért a webshop adataihoz kötöttek, és nem válhatnak teljesen általános, bizonytalan ajánlásokká. Ha nincs pontos terméktalálat, a rendszer inkább általános iránymutatást ad, és jelzi, hogy pontos ajánlathoz további egyeztetés szükséges. Így az asszisztens nem ad magabiztos, de pontatlan választ olyan helyzetekben, ahol a katalógus adatai nem elegendők.

9. Biztonság és adatvédelem

A biztonság egy webes rendszernél nem külön kiegészítő, hanem alapfeltétel. A TDLWebshop esetében külön kellett figyelni a felhasználói adatok és a rendelések kezelésére, az admin jogosultságokra, a kuponokra, a PDF-ek tartalmára, és nem utolsósorban az AI-asszisztenshez kapcsolódó kulcsokra. Mivel a rendszer Firebase-en fut, a biztonsági szabályok kiemelt szerepet kapnak. A kliensoldali ellenőrzés önmagában nem elég, hiszen egy rosszindulatú kliens közvetlenül is megpróbálhatja módosítani az adatbázist. Emiatt a Firestore-rules szerepkör, aktív felhasználói állapot és a dokumentumok szerkezete alapján engedi vagy tiltja az olvasási és írási műveleteket. A titkok kezelése is fontos szempont volt: API-kulcs vagy jelszó nem kerülhet a repóba, ezért az OpenRouter-kulcs Cloudflare Worker secretként van kezelve, a repóban csak a minta-konfiguráció található.

A főbb kockázatokat és a hozzájuk tartozó megoldásokat érdemes konkrétan is összepárosítani. Az admin jogosultság megszerzésére irányuló kísérlet ellen a Firestore-szabályok védenek: minden írási műveletnél ellenőrzik a kérő felhasználó `users/{uid}` dokumentumában szereplő `role` mezőt, így a UI elrejtése nem kerülhető meg egyszerű direkt adatbázis-hívással. Az AI API-kulcs kiszivárgását úgy akadályoztam meg, hogy nem a frontendre tettem, hanem a Cloudflare Worker környezeti változójába (`env.OPENROUTER_KEY`), és a Worker csak a saját TDLWebshop domainről érkező kérést fogadja el. A kuponmanipulációt serveroldali ellenőrzés akadályozza: bár a kedvezmény kiszámítása részben a kliensen történik, a rendelés mentésekor a kuponkód érvényessége és a max. felhasználás külön gyűjteményi olvasással kerül újraellenőrzésre. A PDF-bizonylatban szereplő személyes adatok (név, cím, telefon, e-mail) csak a saját rendelő vásárló és az admin szerepkör számára férhetők hozzá; a fájl URL-je nem nyilvános, és a Firestore-szabályok itt is érvényesülnek. A teljes kockázat-megoldás összefoglaló a 8. táblázatban szerepel.

8. táblázat – Biztonsági minimum és kockázatok

Kockázat	Leírás	Kezelés a projektben	További javaslat
Jogosulatlan admin-művelet	Normál user admin adatot módosítana.	Szerepköralapú rules és frontend-guard.	Custom claim és token revocation éles rendszerben.

Kupon-visszaélés	Kliensoldali kuponmanipuláció.	Kuponvalidáció és dokumentált MVP-korlát.	Szerveroldali újraszámolás.
PDF-adatvédelem	Személyes adatok szerepelnek a bizonylaton.	Csak jogosult felületen generálható.	Letöltési naplózás és tárolási szabály.
AI API-kulcs kiszivárgása	Kulcs frontendbe vagy gitbe kerül.	Cloudflare Worker secret, .env.example.	Rate limit és kvóta.

A részletes jogosultsági szabályok az M2 mellékletben szerepelnek (firestore.rules 294–355).

A jogosultságkezelés teljes kódja az M2 mellékletben található (admin.ts 607–650).

A biztonsági szabályok mellett a CORS-kezelés (Cross-Origin Resource Sharing) is fontos része volt a megoldásnak. A Cloudflare Worker, amely az AI-asszisztens proxyja, expliciten korlátozza, mely domainről érkező kérést fogad el. A logika lényegét az alábbi rövid kódrészlet mutatja:

```
const ALLOWED_ORIGINS = new Set([
  'https://tdlwebshop.web.app',
  'https://tdlwebshop.firebaseio.com',
  'http://localhost:4200',
  'http://127.0.0.1:4200'
]);

function corsHeaders(request) {
  const origin = request.headers.get('Origin') || '';
  const headers = {
    'Access-Control-Allow-Methods': 'POST, OPTIONS',
    'Access-Control-Allow-Headers': 'Content-Type',
    Vary: 'Origin'
  };

  if (ALLOWED_ORIGINS.has(origin)) {
    headers['Access-Control-Allow-Origin'] = origin;
  }

  return headers;
}
```

1. kódrészlet. Cloudflare Worker AI-proxy CORS-szűréssel.

A megoldás lényege, hogy a proxy csak a saját TDLWebshop domain URL-jéről érkező kérést szolgálja ki – minden más Origin esetén üres Allow-Origin headert ad vissza, így a böngésző elutasítja a választ. Erre azért van szükség, mert ha bárki ismeri a proxy URL-jét, egyébként ingyen használhatná az AI-szolgáltatást a saját API-kulcsom terhére. Az OpenRouter-kulcs ezen felül a Worker környezeti változójában (env.OPENROUTER_KEY) tárolódik, nem a forráskódban, így a GitHub repóból sem szivároghat ki. A megoldás nem ad teljes védelmet visszaélés ellen – egy elhatározott támadó hamis Origin headert is küldhet közvetlen HTTP-hívással –, ehhez éles környezetben rate-limit vagy felhasználói tokenhez kötött kvóta is szükséges lenne. Tudatos MVP-korlát, hogy ezeket a következő iterációba terveztem.

9.1. MVP-korlátok biztonsági szempontból

A rendszer szándékosan nem vállalja egy teljes pénzügyi backend szerepét. A bankkártya-elfogadás, a szerveroldali ár-újraszámolás, a készletfoglalás és az ERP-szintű integráció további fejlesztési irányok, nem hibák. Az AI-asszisztensnél a CORS-beállítás önmagában nem véd meg minden visszaéléstől; éles környezetben rate limit, kvóta vagy felhasználóhoz kötött híváskorlátozás tudná tovább erősíteni.

10. Tesztelés

A tesztelés célja az volt, hogy a rendszer fontosabb folyamatai ne csak tervezési szinten, hanem a gyakorlatban is ellenőrizhetők legyenek. Ezt automatikus tesztekkel és kézi ellenőrzéssel vizsgáltam. Az automatikus tesztek a CI-folyamatban is lefutnak, így a GitHub Actions zöld állapota jó visszajelzés arra, hogy a build és a tesztek rendben vannak. A kézi tesztelés során azt vizsgáltam, hogy a felület minden eleme a tervezett módon működik-e: a hibák megfelelően jelennek meg, a rendelési folyamat hibamentesen végigfut, és a rendszer különböző helyzetekben is stabil marad. Külön figyeltem a checkoutra és a rendelés létrejöttére, a státuszváltásra és a hozzá tartozó készletmódosításra, a jogosultságok érvényesítésére, a CSV-importra és az AI-asszisztens válaszaira.

9. táblázat – Tesztelési terv összefoglalása

Tesztazonosító	Folyamat	Elvárt eredmény	Bizonyíték
T1	Regisztráció és belépés	Felhasználó profilja létrejön, tiltott user üzenetet kap.	Kézi teszt
T4	Checkout-validáció	Hibás e-mail/telefon nem enged tovább.	Képernyőkép
T5	Sikeres rendelés	Rendelés létrejön Firestore-ban és adminban látszik.	Kézi teszt
T6	Admin státuszváltás	Audit és készletváltozás rögződik.	Automatikus/kézi
T7	CSV-import	Hibás sorok jelződnek, érvényes sorok menthetők.	Kézi teszt
T8	PDF-bizonylat	PDF tartalmazza az adatokat és nem csúszik össze.	Képernyőkép

T9	AI-asszisztens	Releváns kérdésre domain-válasz, irrelevánsra elhatárolás.	Kézi teszt
T10	CI	GitHub Actions build/test zöld.	Képernyőkép

10.1. Build és CI

A projekt CI-folyamata GitHub Actions segítségével fut, és minden fontosabb commit után ellenőrzi, hogy a rendszer továbbra is buildelhető-e, és a tesztek hibamentesen lefutnak-e. A zöld CI-állapot egyértelmű bizonyíték arra, hogy a projekt nemcsak lokális környezetben működik.

10.2. Kézi tesztjegyzőkönyv

A kézi tesztelés során nem csak a sikeres eseteket vizsgáltam, hanem a hibás bemeneteket és a tiltott műveleteket is. Egy webshopnál nem elég, hogy helyes adatokkal minden működik, azt is látni kell, hogyan reagál a rendszer hibás használat esetén. A checkout oldalon például külön ellenőriztem azt az esetet, amikor a felhasználó hibás e-mail-címet vagy telefonszámot ad meg: a rendszer ilyenkor nem engedi véglegesíteni a rendelést, hanem hibaüzenetet jelenít meg, és jelzi, melyik mezőt kell javítani. A teljes tesztjegyzőkönyv az M1 mellékletben található.

A főbb tesztesetekhez mind a pozitív, mind a negatív vagy hibás bemenetű változatot megnéztem. A checkoutnál a pozitív eset a helyes e-mail-, telefon- és számlázási adatokkal lefutó rendelés, a negatív eset a hibás e-mail-formátum vagy rövid telefonszám, amikor a rendszer hibaüzenettel megakasztja a folyamatot. Az admin műveleteknél a pozitív eset egy bejelentkezett admin státuszváltása, a negatív eset az, amikor egy egyszerű felhasználó próbál Firestore-hívással rendelést módosítani – ezt a Firestore-szabály utasítja el. A CSV-importnál a pozitív eset a megfelelő szerkezetű fájl betöltése, a negatív eset egy rossz elválasztású vagy hiányzó kötelező oszlopú fájl, amelyet a validáció nem enged menteni. Az AI-asszisztensnél pozitív eset egy katalógusban szereplő termékre vonatkozó kérdés, negatív eset egy teljesen irreleváns kérdés (pl. "mennyi az időjárás holnap"), amelyre az asszisztens szándékosan nem ad érdemi választ, hanem visszairányít a katalógusra.

11. Bizonyítékok összefoglalása

Az alábbi táblázat a dolgozat főbb állításait kapcsolja össze konkrét bizonyítékokkal. Minden sorban szerepel egy fontos elem, az, hogy a dolgozatban hol vagy melyik mellékletben jelenik meg, és az is, hogy a rendszer működéséből mit igazol. A táblázat összegyűjti a dolgozat fő érveinek bizonyítási vázát.

Bizonyíték	Hol szerepel / hol ellenőrizhető	Mit igazol
GitHub Actions zöld CI-képernyőkép	Reprodukálhatóság és tesztelés fejezet	A projekt tiszta környezetben is buildelhető és tesztelhető.
Firestore-rules kódrészlet	Biztonsági fejezet	A jogosultságok nem csak kliensoldali, hanem adatbázis szinten is kezeltek.
Checkout-validáció képernyőkép	GUI/UX és tesztelés fejezet	A hibás bemeneteket a rendszer felhasználói szinten is kezeli.
Helyszíni vásárlás képernyőkép	Megvalósítás fejezet	Az admin oldali belső bolti folyamat is működik.
PDF-bizonylat képernyőkép	Megvalósítás és eredmények fejezet	A rendelési adatból strukturált dokumentum készül.
AI-asszisztens képernyőkép	MI-használat és megvalósítás fejezet	A rendszerben működő AI-funkció elkülönül a fejlesztést segítő AI-használattól.

12. Reprodukálhatóság, CI és deploy

Egy szakdolgozati projekt akkor igazán jó, ha nem csak a saját gépemen fut, hanem más környezetben is elindítható. Emiatt vált fontos részévé a README, a minta környezeti változók, a telepítési leírás és a CI-ellenőrzés. A repóban nem maradt érzékeny adat, node_modules-mappa, buildelt fájl vagy géphez kötött konfiguráció. A kódban csak az szerepel, amire a futtatáshoz és a megértéshez szükség van. A titkokat környezeti változóknak vagy szolgáltatói secretként kezelem, nem a forráskódban: az OpenRouter-kulcs például csak a proxy oldalon érhető el. A deploy két részből áll. A frontend Firebase Hostingra kerül, az AI-asszisztens proxyja pedig egy Cloudflare Worker-en fut. Így a kliensoldal soha nem kap közvetlenül kulcsot, és a külső AI-hívások is biztonságosabban kezelhetők.

10. táblázat – Reprodukciós és deploy-lépések

Lépés	Parancs / ellenőrzés	Cél
Függőségek telepítése	npm install	Angular-projekt függőségeinek telepítése.
Build	npm run build && npm test	Frontend buildelhetőségének és tesztjeinek ellenőrzése.
Hosting deploy	firebase deploy --only hosting	Frontend publikálása.
Firestore-rules deploy	firebase deploy --only firestore:rules	Biztonsági szabályok élesítése.
Worker deploy	npx wrangler deploy	OpenRouter-proxy publikálása.

13. Mesterséges intelligencia használata a fejlesztés során

A szakdolgozat készítése során több mesterségesintelligencia-eszközt is használtam, köztük a ChatGPT-t, a Codexet és a Claude.ai-t. Ezeket főként ötletelésre, kódrészletek átgondolására, hibakeresésre, dokumentációs vázlat készítésére, valamint a szövegek nyelvi finomítására alkalmaztam. Az AI-eszközök használata több ponton is gyorsította a munkát, ugyanakkor fontos volt, hogy a kapott javaslatokat minden esetben saját ellenőrzés után használjam fel. A fejlesztés során használt AI-eszközök és a rendszerben működő AI-asszisztens nem ugyanazt a szerepet töltik be: előbbiek a munkát segítették, utóbbi pedig a TDLWebshop egyik funkciója.

Az AI különösen akkor volt hasznos, amikor egy-egy megoldást több szempontból kellett átgondolni, vagy gyorsan kellett alternatívát keresni egy hibás vagy nem megfelelő részlet helyett. Ez főleg a funkciók tervezésénél, a kódlogika átnézésénél és a dokumentáció szerkezetének kialakításánál segített. A táblázatok és diagramok elkészítésében is támogatást adott, elsősorban abban, hogy az információk áttekinthetőbb formában jelenjenek meg, és könnyebb legyen a megfelelő szerkezet kialakítása. A javaslatokat viszont nem lehetett változtatás nélkül átvenni, mert előfordult, hogy túl általánosak voltak, vagy nem illeszkedtek pontosan a projekt felépítéséhez.

A javaslatokat sosem vettem át kritika nélkül. Mindig leellenőriztem, hogy a kód fordul-e, a tesztek lefutnak-e, és hogy a megoldás illeszkedik-e a projekt többi részéhez. Ha valami gyanús volt, akkor a hivatalos dokumentációval is összevettem, vagy egy másik modellel rákérdeztem ugyanarra. Egy ilyen eset volt a Firestore rules írásánál: a javasolt szerepkör-ellenőrzés egy egyszerű `request.auth.token.role === 'admin'` formát használt, ami a deploy során nem dobott hibát, futtatáskor viszont minden jogosult kérés is elhasalt, mert nálam a szerepkör nem a tokenben volt eltárolva. Az emulator logjából láttam, hogy hol bukik el, és végül egy `users/{uid}` dokumentum `role` mezőjéből olvasó lookup-ra cseréltem a szabályt.

Egy másik példa a PDF-bizonylat oldaltöréseinél jött elő. Az AI azt javasolta, hogy a jsPDF táblázatkezelőjének alapbeállításával rajzoljam ki a tételsorokat, és ez egyoldalas rendelésnél tényleg jól is működött. A több tételes vásárlásnál viszont az oldaltörésnél nem jelent meg újra a fejléc, és a layout szétcsúszott. Végül kézzel kellett megírnom az oldaltörés-figyelést és a fejléc újrarajzolását. Ezek a tapasztalatok arra tanítottak meg, hogy az AI legtöbbször a leggyakoribb mintát javasolja, ami nem feltétlenül illik a saját projekt

felépítéséhez. A sablonkódnál és a dokumentációs szövegezésnél a leggyorsabban használható volt, a kritikusabb üzleti logikánál viszont mindig szigorúbb ellenőrzés kellett.

A rendszerben működő AI-asszisztens ettől eltérő célt szolgál. Nem teljes műszaki tanácsadást ad, és nem helyettesíti a szakértői döntést, hanem inkább abban segít, hogy a felhasználó a termékkatalógus alapján gyorsabban eligazodjon. A célja az, hogy az alapvető kérdésekre irányt mutató választ adjon, és segítse a termékek közötti tájékozódást. Ha nincs pontos találat, vagy a rendelkezésre álló adat nem elég biztos válaszhoz, akkor inkább további egyeztetést javasol. Ezzel az asszisztens egyszerre marad hasznos és korlátozott, ami egy szakmai webshop esetében kifejezetten fontos.

Voltak részek, ahol szándékosan nem használtam AI-t. Az MVP-határ meghúzását, a funkciók fontossági sorrendjét és a vállalt korlátok meghatározását saját döntésnek tartottam, mert ezek adják a dolgozat saját érvkészletét. A 14. fejezet összefoglalója és a saját értékelésem szövege is teljes egészében saját megfogalmazásban készült. A biztonsági kockázatok átgondolásánál csak a szintaxishoz kértem segítséget, a kockázati listát és a Firestore rules logikáját magam állítottam össze. Összességében az AI segített gyorsítani a munkát, de nem volt olyan rész, amit nélkülem készített volna. Ha új projektet kezdenék, valószínűleg ugyanígy használnám, csak még szigorúbban néznék rá a kódba kerülő részekre.

11. táblázat – AI-használat és ellenőrzés

AI-használat területe	Használat célja	Ellenőrzés
Ötletelés	Funkciók, MVP-határ és dokumentáció átgondolása.	Konzulensi elvárásokkal való összevetés.
Kódreview	Hibák, jogosultságok, validáció és tranzakciók átnézése.	Kóddolvasás, build és teszt.
Dokumentáció	Fejezetvázlat és biztonsági alapok előkészítése.	Saját átírás és szakmai ellenőrzés.
AI-asszisztens funkció	Katalógushoz kötött válaszadás.	Tesztkérdések kipróbálása.

14. Összefoglalás, korlátok és továbbfejlesztés

A TDLWebshop egy épületgépészeti webshop és adminisztrációs rendszer MVP-je: vásárlói oldal, admin felület, Firestore-adatmodell, jogosultsági szabályok, PDF-bizonylat, CSV-import, készletkezelés és AI-asszisztens együttesen alkotják. A munka egyik legjelentősebb eredménye, hogy ezeket a funkciókat sikerült egységesen összekapcsolni: a vásárlói rendelés nem önmagában jelenik meg, hanem az admin felületen is, státuszt kap, kapcsolódik a PDF-bizonylathoz és a készletlogikához. A helyszíni vásárlás és a mentett vásárló kezelése pedig külön domainértéket adott a rendszernek.

A fejlesztés során voltak nehezítő tényezők. Az egyik, hogy nem fizettem elő a Firebase-re, ezért az ingyenes verzió korlátai miatt sok mindent körütekintőbben kellett megoldani. A PDF-generálással is sok idő elment, mert úgy kellett összerakni, hogy működjön is, közben jól is nézzen ki. A rendszernek vannak tudatos korlátai: nincs benne teljes pénzügyi integráció, a szerveroldali áru-újraszámolás és a készletfoglalás még továbbfejleszthető, az AI-asszisztensnél pedig hasznos lenne a rate limit és a kvóta erősítése. Ezeket inkább továbbfejlesztési iránynak tartom, nem hibának.

Amire a legbüszkébb vagyok, az a kinézet és az, hogy mennyire összetett admin oldalt és jogosultságkezelést sikerült létrehozni. Ezek sokat hozzátesznek ahhoz, hogy a rendszer ne csak egy egyszerű bemutató legyen, hanem összerakott, használható projekt. Ha újra kezdeném, valószínűleg több időt hagynék a tervezésre, és az elején jobban átgondolnám, milyen újdonságokat lehetne még beépíteni; ötletem most is van arra, merre lehetne továbbfejleszteni.

Irodalomjegyzék

- [1] Angular – Hivatalos dokumentáció. <https://angular.dev/>
- [2] Firebase – Hivatalos dokumentáció. <https://firebase.google.com/docs>
- [3] Cloud Firestore Security Rules – Hivatalos dokumentáció.
<https://firebase.google.com/docs/firestore/security/get-started>
- [4] Cloudflare Workers – Hivatalos dokumentáció.
<https://developers.cloudflare.com/workers/>
- [5] OpenRouter – Hivatalos dokumentáció. <https://openrouter.ai/docs>
- [6] GitHub Actions – Hivatalos dokumentáció. <https://docs.github.com/actions>

Mellékletek

M1. Kézi tesztjegyzőkönyv kitöltött állapotban (repó: docs/testing/manual-test-log.md).

M2. Kiemelt kódrészletek a repóban: checkout (src/pages/checkout/checkout.ts), OrderService (src/app/services/order.service.ts), Firestore rules (firestore.rules), InvoiceService (src/app/services/invoice.service.ts), AI proxy (workers/openrouter-proxy/src/index.js).

M2.1. ábra: Útvonalkezelés és lazy loading megvalósítása az Angular alkalmazásban. A kódrészlet azt mutatja be, hogyan vannak elkülönítve a publikus, vásárlói és adminisztrációs felületek, valamint hogyan töltődnek be az egyes oldalak csak szükség esetén.

M2.1 - Útvonalkezeles es lazy loading

```
src/app/app.routes.ts | 5-21. sor
5   export const routes: Routes = [
6     { path: '', loadComponent: () => import('../pages/home/home').then(m => m.Home) },
7     { path: 'products', loadComponent: () => import('../pages/products/products').then(m => m.Products) },
8     { path: 'products/:id', loadComponent: () => import('../pages/product-details/product-details').then(m =>
9       m.ProductDetails) },
10    { path: 'categories', loadComponent: () => import('../pages/categories/categories').then(m => m.Categories) },
11    { path: 'contact', loadComponent: () => import('../pages/contact/contact').then(m => m.Contact) },
12    { path: 'wishlist', loadComponent: () => import('../pages/wishlist/wishlist').then(m => m.Wishlist) },
13    { path: 'cart', loadComponent: () => import('../pages/cart/cart').then(m => m.Cart) },
14    { path: 'checkout', loadComponent: () => import('../pages/checkout/checkout').then(m => m.Checkout) },
15    { path: 'order-success', loadComponent: () => import('../pages/order-success/order-success').then(m =>
16      m.OrderSuccess) },
17    { path: 'profile', loadComponent: () => import('../pages/profile/profile').then(m => m.Profile) },
18    { path: 'my-orders', redirectTo: 'profile' },
19    { path: 'login', loadComponent: () => import('../pages/login/login').then(m => m.Login) },
20    { path: 'register', loadComponent: () => import('../pages/register/register').then(m => m.Register) },
21    // Admin oldalra csak jogosult user mehet be.
22    { path: 'admin', loadComponent: () => import('../pages/admin/admin').then(m => m.Admin), canActivate:
23      [adminGuard] },
24    { path: '**', redirectTo: '' }
```

M2.2. ábra: Admin útvonalvédő működése. A kódrészlet szemlélteti, hogy az admin felület csak megfelelő jogosultságú felhasználó számára érhető el.

M2.2 - Admin route guard

```
src/app/guards/admin.guard.ts | 7-23. sor
7   export const adminGuard: CanActivateFn = async (_route, state) => {
8     const authService = inject(AuthService);
9     const router = inject(Router);
10
11    await authService.waitForAuthReady();
12
13    if (authService.isCurrentUserStaff()) {
14      return true;
15    }
16
17    return router.createUrlTree(['/login'], {
18      queryParams: {
19        redirectTo: state.url,
20        adminOnly: '1'
21      }
22    });
23  };

```

M2.3. ábra: Felhasználói szerepkör ellenőrzése. A részlet azt mutatja, hogyan különbözteti meg a rendszer a vásárlói, dolgozói és adminisztrátori jogosultságokat.

M2.3 - Szerepkör-ellenorzes

```
src/app/services/auth.service.ts | 104-122. sor
104    return this.initializedPromise;
105  }
106
107  isAdminEmail(email?: string | null) {
108    if (!email) {
109      return false;
110    }
111
112    return environment.adminEmails.includes(email.toLowerCase());
113  }
114
115  isCurrentUserAdmin() {
116    return this.isAdminEmail(this.getUser()?.email) || this.getProfile()?.role === 'admin';
117  }
118
119  isCurrentUserEmployee() {
120    return this.getProfile()?.role === 'employee';
121  }
122

```

M2.4a. ábra: Firestore admin és aktív felhasználó ellenőrző függvények. A részlet bemutatja, hogyan azonosítja a szabályrendszer az admin e-mail címeket, a bejelentkezett felhasználót, valamint azt, hogy a felhasználói dokumentum alapján aktívnek tekinthető-e a user.

```
M2.4a - Firestore admin es aktiv user helper

1  rules_version = '2';
2
3  service cloud.firestore {
4    match /databases/{database}/documents {
5      function adminEmailList() {
6        return [
7          'admin@tdlwebshop.hu',
8          'teszt@tdlwebshop.hu'
9        ];
10     }
11
12     function isAdminEmail(email) {
13       return email is string && email in adminEmailList();
14     }
15
16     function isSignedIn() {
17       return request.auth != null;
18     }
19
20     function userDocExists() {
21       return isSignedIn()
22         && exists(/databases/{database}/documents/users/{request.auth.uid});
23     }
24
25     function isActiveUser() {
26       // Tiltott felhasználó érvényes tokennel se tudjon írni, új regisztrációnál viszont még nincs user
27       // dokumentum.
28       return isSignedIn()
29         && (
30           !userDocExists()
31           || get(/databases/{database}/documents/users/{request.auth.uid}).data.disabled != true
32         );
33     }
34   }
35 }
```

M2.4b. ábra: Firestore admin jogosultság ellenőrzése. A kódrészlet azt szemlélteti, hogy a rendszer tokenben szereplő admin e-mail vagy felhasználói dokumentumban tárolt admin szerepkör alapján engedélyezi az admin műveleteket.

```
M2.4b - Firestore admin jogosultsag helper

1  rules_version = '2';
2
3  service cloud.firestore {
4    match /databases/{database}/documents {
5      function isAdmin() {
6        return isSignedIn()
7          && (
8            request.auth.token.email != null
9            && request.auth.token.email in adminEmailList()
10          )
11          || (
12            exists(/databases/{database}/documents/users/{request.auth.uid})
13            && get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'admin'
14          );
15      }
16    }
17  }
```

M2.4c. ábra: Firestore termék- és tartalomkezelési szabályok. A részlet bemutatja, hogy a termékek nyilvánosan olvashatók, de létrehozásuk, módosításuk és törlésük admin vagy megfelelő dolgozói jogosultsághoz kötött.

```
M2.4c - Firestore termék és tartalom szabalyok

firestore.rules | 288-309. sor
288   match /products/{productId} {
289     allow read: if true;
290     allow create: if (isAdmin() || employeeCanManageProducts()) && validProductPayload();
291     allow update: if isAdmin() || employeeCanManageProducts();
292     allow delete: if isAdmin();
293   }
294
295   match /installerPackages/{packageId} {
296     allow read: if true;
297     allow create, update, delete: if isAdmin();
298   }
299
300   match /news/{newsId} {
301     allow read: if true;
302     allow create, update: if isAdmin() && validNewsPayload();
303     allow delete: if isAdmin();
304   }
305
306   match /newsletterSubscribers/{subscriberId} {
307     allow create: if validNewsletterSubscriberPayload();
308     allow read, update, delete: if isAdmin();
309   }
```

M2.4d. ábra: Firestore rendeléskezelési szabályok. A kódrészlet azt mutatja, hogyan korlátozza a rendszer a rendelések létrehozását, olvasását, módosítását és törlését admin, dolgozói vagy rendelőtulajdonosi jogosultság alapján.

```
M2.4d - Firestore rendeles szabalyok

firestore.rules | 311-316. sor
311   match /orders/{orderId} {
312     allow create: if isAdmin() || employeeCanRecordSales() || validOrderCreate();
313     allow read: if isAdmin() || employeeCanRecordSales() || isOrderOwner();
314     allow update: if isAdmin() || employeeCanRecordSales();
315     allow delete: if isAdmin();
316   }
```

M2.5. ábra: Rendelés adatmodellje. A részletben látható, hogy a rendelés milyen mezőkből épül fel, beleértve a vásárlói adatokat, tételeket, összegeket és státuszokat .

M2.5 - Rendelés adatmodell

```
src/app/models/order.model.ts | 3-63.sor
3 // Rendelés adatmodell Firestore-hoz és UI-hoz.
4 export interface Order {
5   id?: string;
6   userId?: string;
7   customerName: string;
8   customerEmail: string;
9   customerPhone: string;
10  shipping: {
11    zip: string;
12    city: string;
13    address: string;
14  };
15  shippingMethod?: {
16    id: string;
17    label: string;
18    fee: number;
19    eta: string;
20  };
21  billing: {
22    sameAsShipping: boolean;
23    name: string;
24    zip: string;
25    city: string;
26    address: string;
27  };
28  paymentMethod?: {
29    id: string;
30    label: string;
31    fee: number;
32    dueDays?: number;
33    dueAt?: number;
34  };
35  business?: {
36    isBusinessBuyer: boolean;
37    companyName?: string;
38    taxNumber?: string;
39  };
40  salesChannel?: 'web' | 'local-admin';
41  pickupAt?: number;
42  invoiceNumber?: string;
43  invoicedAt?: number;
44  appliedCoupon?: {
45    code: string;
46    description: string;
47    discount: number;
48    type: 'percent' | 'fixed' | 'shipping';
49  };
50  pricing?: {
51    subtotal: number;
52    shippingFee: number;
53    paymentFee: number;
54    discount: number;
55    total: number;
56  };
57  couponCode: string;
58  comment: string;
59  items: CartItem[];
60  total: number;
61  status: string;
62  createdAt?: number;
63 }
```

M2.6a. ábra: Checkout validáció és összegszámítás. A kódrészlet azt szemlélteti, hogyan ellenőrzi a rendszer a rendeléshez szükséges adatokat, majd hogyan számolja ki a szállítási díjat, kedvezményt és végösszeget.

M2.6a - Checkout validacio es osszegezes

```
src/pages/checkout/checkout.ts | 457-486. sor
457   const validationErrors = this.validateOrderForm(sanitized);
458
459   if (validationErrors.length > 0) {
460     this.orderError = 'A rendelés nem küldhető el, javítsd a hibákat.';
461     this.orderValidationErrors = validationErrors;
462     this.toastService.error(' adatok', this.orderError);
463     return;
464   }
465
466   this.orderLoading = true;
467
468   try {
469     if (this.authService.isCurrentUserDisabled()) {
470       this.orderError = 'A profilod le van tiltva, ezért rendelést nem lehet leadni.';
471       this.toastService.error(' nem el', this.orderError);
472       return;
473     }
474
475     const currentUser = this.authService.getUser();
476     const shippingMethod = this.activeShippingMethod;
477     const paymentMethod = this.activePaymentMethod;
478     const pickupAt =
479       shippingMethod.id === 'pickup'
480         ? this.parsePickupDateTime(sanitized.pickupDateTime)
481         : null;
482     const subtotal = this.subtotal;
483     const shippingFee = shippingMethod.fee;
484     const paymentFee = paymentMethod.fee;
485     const coupon = this.coupon;
486     const discount = this.discount;
```

M2.6b. ábra: Rendelés objektum összeállítása. A részlet bemutatja, hogy a checkout folyamat során a rendszer milyen vásárlói, szállítási, számlázási, fizetési és kuponadatokat rendez egy menthető rendelési objektumba.

M2.6b - Rendeles objektum osszeallitasa

```
src/pages/checkout/checkout.ts | 488-545. sor
488
489     const orderRef = await this.orderService.addOrder({
490       userId: currentUser?.uid,
491       customerName: sanitized.customerName,
492       customerEmail: sanitized.customerEmail,
493       customerPhone: sanitized.customerPhone,
494       shipping: {
495         zip: sanitized.shippingZip,
496         city: sanitized.shippingCity,
497         address: sanitized.shippingAddress
498       },
499       shippingMethod: {
500         id: shippingMethod.id,
501         label: shippingMethod.label,
502         fee: shippingFee,
503         eta: shippingMethod.eta
504       },
505       billing: {
506         sameAsShipping: this.billingSameAsShipping,
507         name: this.billingSameAsShipping ? sanitized.customerName : sanitized.billingName,
508         zip: this.billingSameAsShipping ? sanitized.shippingZip : sanitized.billingZip,
509         city: this.billingSameAsShipping ? sanitized.shippingCity : sanitized.billingCity,
510         address: this.billingSameAsShipping ? sanitized.shippingAddress : sanitized.billingAddress
511       },
512       paymentMethod: {
513         id: paymentMethod.id,
514         label: paymentMethod.label,
515         fee: paymentFee
516       },
517       pickupAt: pickupAt || undefined,
518       business: {
519         isBusinessBuyer: this.isBusinessBuyer,
520         companyName: this.isBusinessBuyer ? (sanitized.businessCompanyName || sanitized.customerName) :
           undefined,
521         taxNumber: this.isBusinessBuyer ? sanitized.businessTaxNumber : undefined
522       },
523       salesChannel: 'web',
524       appliedCoupon: coupon
525       ? {
526         code: coupon.code,
527         description: coupon.description,
528         discount: coupon.discount,
529         type: coupon.type
530       }
531       : undefined,
532       pricing: {
533         subtotal,
534         shippingFee,
535         paymentFee,
536         discount,
537         total
538       },
539       couponCode: sanitized.couponCode,
540       comment: sanitized.orderComment,
541       items: items,
542       total,
543       status: 'uj'
544     });
545
```

M2.6c. ábra: Rendelés összegzés és profilmentés. A kódrészlet azt mutatja, hogyan készül el a sikeres rendeléshez szükséges összegző adatcsomag, illetve hogyan kapcsolódik a rendelés mentése a bejelentkezett felhasználó profiladatainak frissítéséhez.

M2.6c - Rendeles osszegzes es profilmentes

```
src/pages/checkout/checkout.ts | 546-618.sor
546   const orderSummary: OrderSuccessSummary = {
547     orderId: orderRef.id,
548     customerName: sanitized.customerName,
549     customerEmail: sanitized.customerEmail,
550     customerPhone: sanitized.customerPhone,
551     shippingMethodLabel: shippingMethod.label,
552     paymentMethodLabel: paymentMethod.label,
553     pickupAt: pickupAt || undefined,
554     subtotal,
555     shippingFee,
556     paymentFee,
557     discount,
558     couponCode: coupon?.code,
559     couponDescription: coupon?.description,
560     total,
561     items: items.map(item => ({
562       name: item.name,
563       quantity: item.quantity,
564       price: item.price,
565       image: item.image
566     }))
567   };
568
569   sessionStorage.setItem('lastOrderSummary', JSON.stringify(orderSummary));
570
571   if (currentUser?.uid) {
572     try {
573       await this.userService.attachOrderToUser(currentUser.uid, orderRef.id);
574     } catch (userOrderLinkError) {
575       this.monitoringService.capture('checkout-user-order-link', userOrderLinkError, {
576         orderId: orderRef.id,
577         uid: currentUser.uid
578       });
579     }
580
581     try {
582       await this.customerDirectoryService.upsertProfileForUser(
583         currentUser.uid,
584         sanitized.customerEmail,
585         {
586           type: this.isBusinessBuyer ? 'company' : 'private',
587           name: sanitized.customerName,
588           phone: sanitized.customerPhone,
589           companyName: this.isBusinessBuyer ? (sanitized.businessCompanyName || sanitized.customerName) :
590             '',
591           taxNumber: this.isBusinessBuyer ? sanitized.businessTaxNumber : ''
592         }
593       );
594     } catch (profileError) {
595       this.monitoringService.capture('checkout-customer-profile-upsert', profileError, {
596         orderId: orderRef.id,
597         email: sanitized.customerEmail,
598         mode: 'user'
599       });
600     }
601   } else {
602     try {
603       await this.customerDirectoryService.upsertGuestProfileByEmail({
604         type: this.isBusinessBuyer ? 'company' : 'private',
605         name: sanitized.customerName,
606         email: sanitized.customerEmail,
607         phone: sanitized.customerPhone,
608         companyName: this.isBusinessBuyer ? (sanitized.businessCompanyName || sanitized.customerName) : '',
609         taxNumber: this.isBusinessBuyer ? sanitized.businessTaxNumber : ''
610       });
611     } catch (guestProfileError) {
612       this.monitoringService.capture('checkout-customer-profile-upsert', guestProfileError, {
613         orderId: orderRef.id,
614         email: sanitized.customerEmail,
615         mode: 'guest'
616       });
617     }
618   }
```

M2.7a. ábra: Checkout űrlaphibák gyűjtése. A részlet bemutatja, hogyan ellenőrzi a checkout oldal a kötelező mezőket, az e-mail címet, a telefonszámot, az adószámot és a szállítási adatokat.

```
src/pages/checkout/checkout.ts | 659-693 sor
659 private validateOrderForm(form: ReturnType<Checkout['getSanitizedForm']>): string[] {
660   // Kliens oldali ellenőrzés az alap hibák kiszűréséhez.
661   const errors: string[] = [];
662   const zipPattern = /^d{4}$/;
663   const taxPattern = /^[d-]{8,15}$/;
664
665   if (!form.customerName) {
666     errors.push('A teljes ');
667   }
668
669   if (!isValidEmail(form.customerEmail)) {
670     errors.push('Adj meg email .');
671   }
672
673   if (!isValidPhone(form.customerPhone)) {
674     errors.push('Adj meg (8-15 , pl. +36 30 123 4567).');
675   }
676
677   if (this.isBusinessBuyer) {
678     if (!form.businessTaxNumber || !taxPattern.test(form.businessTaxNumber)) {
679       errors.push(' vagy .');
680     }
681   }
682
683   if (!zipPattern.test(form.shippingZip)) {
684     errors.push('A 4 legyen. ');
685   }
686
687   if (!form.shippingCity) {
688     errors.push('A ');
689   }
690
691   if (form.shippingAddress.length < 5) {
692     errors.push('A ');
693   }
```

M2.7b. ábra: E-mail validációs segédfüggvény. A kódrészlet azt mutatja, hogyan szűri ki a rendszer az üres, túl hosszú, hibásan tagolt vagy érvénytelen domainnel rendelkező e-mail címeket.

```
src/app/utils/form-validator.ts | 1-38 sor
1 export function isValidEmail(value: string): boolean {
2   const email = value.trim().toLowerCase();
3   if (!email || email.length > 254 || !/s/.test(email)) {
4     return false;
5   }
6
7   const parts = email.split('@');
8   if (parts.length !== 2) {
9     return false;
10  }
11
12  const [localPart, domain] = parts;
13  if (
14    !localPart ||
15    !domain ||
16    localPart.length > 64 ||
17    localPart.startsWith('.') ||
18    localPart.endsWith('.') ||
19    localPart.includes('.') ||
20    domain.startsWith('.') ||
21    domain.endsWith('.') ||
22    domain.includes('.')
23  ) {
24    return false;
25  }
26
27  const labels = domain.split('.');
28  if (labels.length < 2 || labels.some(label => !label || label.startsWith('-') || label.endsWith('-'))) {
29    return false;
30  }
31
32  const tld = labels[labels.length - 1];
33  if (!/^[a-z]{2,}$/i.test(tld)) {
34    return false;
35  }
36
37  return /^[a-z0-9.!#$%&'*/=?^_`{|}~]+@[a-z0-9-]+(?:\.[a-z0-9-]+)+$/i.test(email);
38 }
```

M2.7c. ábra: Telefonszám validációs segédfüggvény. A részlet bemutatja, hogyan kezeli a rendszer a magyar telefonszámformátumokat, a körzetszámokat és a nem megengedett karaktereket.

M2.7c - Telefonszám validacio helper

```
src/app/utils/form-validator.ts | 40-89 sor
40  const HUNGARIAN_LANDLINE_AREA_CODES = new Set([
41    '22', '23', '24', '25', '26', '27', '28', '29',
42    '32', '33', '34', '35', '36', '37',
43    '42', '44', '45', '46', '47', '48', '49',
44    '52', '53', '54', '56', '57', '59',
45    '62', '63', '66', '68', '69',
46    '72', '73', '74', '75', '76', '77', '78', '79',
47    '82', '83', '84', '85', '87', '88', '89',
48    '92', '93', '94', '95', '96', '99'
49  ]);
50
51  function isValidHungarianNationalNumber(nationalNumber: string): boolean {
52    if (/^(20|30|31|50|70)\d{7}$/.test(nationalNumber)) {
53      return true;
54    }
55
56    if (/^1\d{7}$/.test(nationalNumber)) {
57      return true;
58    }
59
60    const areaCode = nationalNumber.slice(0, 2);
61    return HUNGARIAN_LANDLINE_AREA_CODES.has(areaCode) && /^d{8}$/.test(nationalNumber);
62  }
63
64  export function isValidPhone(value: string): boolean {
65    const phone = value.trim();
66    if (!phone) {
67      return false;
68    }
69
70    if (!/^\+?\d\s(.+)$/.test(phone)) {
71      return false;
72    }
73
74    const compact = phone.replace(/[ \s()-.]/g, '');
75    if (!/^\+?\d{8,15}$/.test(compact)) {
76      return false;
77    }
78
79    const digits = compact.replace(/^\+/, '');
80    if (digits.startsWith('36')) {
81      return isValidHungarianNationalNumber(digits.slice(2));
82    }
83
84    if (digits.startsWith('06')) {
85      return isValidHungarianNationalNumber(digits.slice(2));
86    }
87
88    return isValidHungarianNationalNumber(digits);
89  }
```

M2.8a. ábra: Rendelésstátusz tranzakció indítása. A kódrészlet bemutatja, hogyan keresi meg a rendszer a módosítandó rendelést, hogyan ellenőrzi az aktuális státuszt, és mikor szükséges készletkorrekciót végezni.

```
M2.8a - Rendelésstátusz tranzakció indítása

src/app/services/order.service.ts | 41-70. sor
41  async updateOrderStatusWithAudit(params: {
42    orderId: string;
43    fromStatus: string;
44    toStatus: string;
45    actorUid?: string;
46    actorEmail?: string;
47    note?: string;
48  }): Promise<void> {
49    // Státuszfrissítés + audit napló + készletkorrekció egy tranzakcióban.
50    const orderRef = doc(db, 'orders', params.orderId);
51    const auditRef = doc(this.orderStatusAuditCollection);
52    const changedAt = Date.now();
53
54    await runTransaction(db, async transaction => {
55      const orderSnap = await transaction.get(orderRef);
56      if (!orderSnap.exists()) {
57        throw new Error('order-not-found');
58      }
59
60      const orderData = orderSnap.data() as Order;
61      const currentStatus = orderData.status || params.fromStatus;
62      if (currentStatus === params.toStatus) {
63        return;
64      }
65
66      const wasCompleted = currentStatus === 'teljesitve';
67      const willBeCompleted = params.toStatus === 'teljesitve';
68      const shouldAdjustStock = wasCompleted !== willBeCompleted;
69      const direction = willBeCompleted ? -1 : 1;
70      // Csak akkor nyúlunk készlethez, ha a rendelés belép vagy kilep a teljesített állapotból.
```

M2.8b. ábra: Készletkorrekció státuszváltáskor. A részlet azt szemlélteti, hogyan olvassa ki a rendszer a rendelés tételeihez tartozó termékeket, hogyan számolja ki a készletváltozást, és hogyan akadályozza meg a negatív készlet kialakulását.

```
M2.8b - Készletkorrekció státuszváltáskor

src/app/services/order.service.ts | 72-116. sor
72  const stockUpdates: Array<{
73    ref: ReturnType<typeof doc>;
74    quantity: number;
75    previousStock: string;
76    delta: number;
77    itemName: string;
78  }> = [];
79
80  if (shouldAdjustStock) {
81    const items = Array.isArray(orderData.items) ? orderData.items : [];
82
83    for (const item of items) {
84      if (!item.firestoreId) {
85        continue;
86      }
87
88      const productRef = doc(db, 'products', item.firestoreId);
89      const productSnap = await transaction.get(productRef);
90      if (!productSnap.exists()) {
91        continue;
92      }
93
94      const productData = productSnap.data() as { stockQuantity?: number; stock?: string };
95      stockUpdates.push({
96        ref: productRef,
97        quantity: Math.max(0, Number(productData.stockQuantity) || 0),
98        previousStock: productData.stock || 'Készleten',
99        delta: Math.max(0, Number(item.quantity) || 0) * direction,
100        itemName: item.name || item.firestoreId
101      });
102    }
103  }
104
105  for (const update of stockUpdates) {
106    const nextQuantityRaw = update.quantity + update.delta;
107    if (update.delta < 0 && nextQuantityRaw < 0) {
108      throw new Error('insufficient-stock:${update.itemName}');
109    }
110
111    const nextQuantity = Math.max(0, nextQuantityRaw);
112    transaction.update(update.ref, {
113      stockQuantity: nextQuantity,
114      stock: this.resolveStockLabel(nextQuantity, update.previousStock)
115    });
116  }
```

M2.8c. ábra: Rendelésfrissítés és audit napló. A kódrészlet bemutatja, hogyan menti a rendszer az új rendelésszámot, valamint hogyan rögzíti az admin műveletet az audit naplóban visszakövethetőség céljából.

```
M2.8c - Rendelesfrissites es audit naplo

src/app/services/order.service.ts | 118-134. sor
118     transaction.update(orderRef, {
119       status: params.toStatus,
120       statusUpdatedAt: changedAt,
121       updatedAt: changedAt
122     });
123
124     transaction.set(auditRef, {
125       orderId: params.orderId,
126       fromStatus: currentStatus,
127       toStatus: params.toStatus,
128       changedAt,
129       changedByUid: params.actorUid || '',
130       changedByEmail: params.actorEmail || '',
131       note: params.note || '',
132       source: 'admin-panel'
133     });
134   });
```

M2.9. ábra: Admin rendelésszám-váltás meghívása a felületről. A részlet azt mutatja, hogyan indítja el az admin felület a rendelés állapotának módosítását.

```
M2.9 - Admin statuszvaltas meghivasa

src/pages/admin/admin.ts | 1577-1614. sor
1577     const normalized = (value || '').trim().toLowerCase();
1578     return ['1', 'true', 'igen', 'yes'].includes(normalized);
1579   }
1580
1581   private resolveStockByQuantity(quantity: number): string {
1582     if (quantity <= 0) {
1583       return 'Nincs keszleten';
1584     }
1585
1586     if (quantity <= environment.lowStockThreshold) {
1587       return 'Szallithato';
1588     }
1589
1590     return 'Keszleten';
1591   }
1592
1593   formatDate(timestamp?: number): string {
1594     if (!timestamp) {
1595       return 'Nincs datum';
1596     }
1597
1598     return new Date(timestamp).toLocaleString('hu-HU');
1599   }
1600
1601   getOrderItemCount(order: Order): number {
1602     return order.items.reduce((sum, item) => sum + item.quantity, 0);
1603   }
1604
1605   getChannelPercent(count: number): number {
1606     const total = this.businessReport.webOrderCount + this.businessReport.localOrderCount;
1607     if (total === 0) {
1608       return 0;
1609     }
1610
1611     return Math.round((count / total) * 100);
1612   }
1613
1614   setOrderListView(view: 'active' | 'completed' | 'local'): void {
```

M2.10a. ábra: Helyszíni vásárlás összeg- és kedvezménycímkei. A részlet bemutatja, hogyan számolja ki az admin felület a helyszíni vásárlás részösszegét, kedvezményét és végösszegét.

```
■ M2.10a - Helyszíni vasarlas osszeg es cimke

src/pages/admin/admin.ts | 2335-2361. sor
2335 }
2336
2337 get localSaleDiscountPercent(): number {
2338   if (this.localSaleIsBusinessBuyer) {
2339     return 10;
2340   }
2341
2342   const email = this.getLocalSaleEmail();
2343   return this.getLoyaltyDiscountPercentByEmail(email);
2344 }
2345
2346 get localSaleDiscountAmount(): number {
2347   return Math.round((this.localSaleSubtotal * this.localSaleDiscountPercent) / 100);
2348 }
2349
2350 get localSaleTotal(): number {
2351   return Math.max(0, this.localSaleSubtotal - this.localSaleDiscountAmount);
2352 }
2353
2354 getLocalSaleDiscountLabel(): string {
2355   if (this.localSaleDiscountPercent <= 0) {
2356     return '';
2357   }
2358
2359   return this.localSaleIsBusinessBuyer
2360     ? 'Nagyker kedvezmény (${this.localSaleDiscountPercent}%)'
2361     : 'Törzsvásárlói kedvezmény (${this.localSaleDiscountPercent}%);
```

M2.10b. ábra: Mentett vásárló előző rendeléseinek kezelése. A kódrészlet azt mutatja, hogyan keresi vissza a rendszer a kiválasztott vásárló korábbi rendeléseit, és hogyan számolja ki az előzmények alapján járó kedvezményt.

```
■ M2.10b - Mentett vasarlo elozo rendelesek

src/pages/admin/admin.ts | 2363-2398. sor
2363
2364 getCustomerProfileOrders(profile: CustomerProfile | null | undefined): Order[] {
2365   const email = (profile?.email || '').trim().toLowerCase();
2366   if (!email) {
2367     return [];
2368   }
2369
2370   return this.orders
2371     .filter(order => (order.customerEmail || '').trim().toLowerCase() === email)
2372     .sort((left, right) => (right.createdAt || 0) - (left.createdAt || 0));
2373 }
2374
2375 getCustomerProfileTotalSpent(profile: CustomerProfile | null | undefined): number {
2376   return this.getCustomerProfileOrders(profile)
2377     .filter(order => order.status === 'teljesitve' || order.salesChannel === 'local-admin')
2378     .reduce((sum, order) => sum + Math.max(0, Number(order.total) || 0), 0);
2379 }
2380
2381 getCustomerProfileDiscountPercent(profile: CustomerProfile | null | undefined): number {
2382   if (profile?.type === 'company') {
2383     return 10;
2384   }
2385
2386   return this.getLoyaltyDiscountPercentByEmail(profile?.email || '');
2387 }
2388
2389 getCustomerProfileDiscountLabel(profile: CustomerProfile | null | undefined): string {
2390   const percent = this.getCustomerProfileDiscountPercent(profile);
2391   if (percent <= 0) {
2392     return 'Nincs kedvezmény';
2393   }
2394
2395   return profile?.type === 'company'
2396     ? 'Nagyker ${percent}%'
2397     : 'Törzsvásárló ${percent}%';
2398 }
```

M2.10c. ábra: Mentett vásárló kiválasztása és kedvezménylogika. A részlet bemutatja, hogyan nyílik meg a vásárlói előzményablak, illetve hogyan dönt a rendszer a nagykereskedelmi vagy törzsvásárlói kedvezményről.

```
src/pages/admin/admin.ts | 2400-2431. sor
2400   openCustomerHistory(profile: CustomerProfile): void {
2401     this.selectedCustomerHistoryProfile = profile;
2402     this.customerHistoryOpen = true;
2403   }
2404
2405   closeCustomerHistory(): void {
2406     this.customerHistoryOpen = false;
2407     this.selectedCustomerHistoryProfile = null;
2408   }
2409
2410   private getLoyaltyDiscountPercentByEmail(email: string): number {
2411     const normalizedEmail = (email || '').trim().toLowerCase();
2412     if (!normalizedEmail) {
2413       return 0;
2414     }
2415
2416     const totalSpent = this.orders
2417       .filter(order =>
2418         (order.customerEmail || '').trim().toLowerCase() === normalizedEmail
2419         && (order.status === 'teljesitve' || order.salesChannel === 'local-admin')
2420       )
2421       .reduce((sum, order) => sum + Math.max(0, Number(order.total) || 0), 0);
2422
2423     if (totalSpent >= 250000) {
2424       return 5;
2425     }
2426
2427     if (totalSpent >= 100000) {
2428       return 3;
2429     }
2430
2431     return 0;
```

M2.10d. ábra: Helyszíni vásárlás elővalidációja. A kódrészlet azt mutatja, hogyan ellenőrzi a rendszer a belső jogosultságot, a kosár tételeit, a vásárlói adatokat, az e-mail címet, a telefonszámot és az adószámot.

```
M2.10d - Helyszíni vasarlas elovalidacio

src/pages/admin/admin.ts | 2433-2478. sor
2433
2434   async submitLocalSale(): Promise<void> {
2435     // Helyszíni eladas teljes folyamata:
2436     // rendelés mentes + készlet frissítés + számla letöltés.
2437     this.localSaleError = '';
2438     this.localSaleSuccess = '';
2439
2440     if (!this.canRecordSales()) {
2441       this.localSaleError = 'Ehhez a művelethez belső jogosultság kell.';
2442       return;
2443     }
2444
2445     if (this.localSaleLines.length === 0) {
2446       this.localSaleError = 'Adj hozzá legalább egy tételt a helyszíni vásárláshoz.';
2447       return;
2448     }
2449
2450     const blockedProfile = this.getBlockedLocalSaleProfile();
2451     if (blockedProfile) {
2452       this.localSaleError = `A le van tiltva: ${blockedProfile.name}. nem`;
2453       return;
2454     }
2455
2456     if (!isValidEmail(this.getLocalSaleEmail())) {
2457       this.localSaleError = 'A helyszíni vásárláshoz adj meg érvényes email címet.';
2458       return;
2459     }
2460
2461     if (!isValidPhone(this.getLocalSalePhone())) {
2462       this.localSaleError = 'A helyszíni vásárláshoz adj meg érvényes telefonszámot (8-15 számjegy).';
2463       return;
2464     }
2465
2466     if (this.localSaleIsBusinessBuyer && !this.localSaleTaxNumber.trim()) {
2467       this.localSaleError = 'Céges vagy szerelői vásárlásnál adószám megadása kötelező.';
2468       return;
2469     }
2470
2471     for (const line of this.localSaleLines) {
2472       const available = this.getAvailableStockForProduct(line.productId);
2473       if (line.quantity > available) {
2474         this.localSaleError = `Nincs elég szabad készlet: ${line.name} (elérhető: ${available} db).`;
2475         return;
2476       }
2477     }
2478
```

M2.10e. ábra: Helyszíni vásárlás tételeinek és összegeinek összeállítása. A részlet bemutatja, hogyan alakítja át a felület a kiválasztott termékeket rendelési tételekké, és hogyan számolja ki a fizetendő végösszeget.

```
M2.10e - Helyszíni tetelek es osszegek

src/pages/admin/admin.ts | 2480-2498. sor
2480
2481   try {
2482     const items: CartItem[] = this.localSaleLines.map((line, index) => ({
2483       id: Date.now() + index,
2484       firestoreId: line.productId,
2485       sku: line.sku,
2486       name: line.name,
2487       price: line.price,
2488       image: line.image,
2489       quantity: line.quantity
2490     }));
2491
2492     const subtotal = this.localSaleSubtotal;
2493     const discount = this.localSaleDiscountAmount;
2494     const total = this.localSaleTotal;
2495     const paymentLabel = this.getLocalSalePaymentLabel();
2496     const paymentDueAt = this.getLocalSalePaymentDueAt();
2497
2498     if (this.localSaleSaveCustomerForLater) {
```

M2.10f. ábra: Helyszíni rendelés objektumának létrehozása. A kódrészlet bemutatja, milyen vásárlói, szállítási, számlázási, fizetési és kedvezményadatokkal jön létre a helyszíni vásárlás rendelési rekordja.

```
M2.10f - Helyszíni rendelés objektum

src/pages/admin/admin.ts | 2500-2549. sor
2500 }
2501
2502 const orderRef = await this.orderService.createLocalSaleOrder({
2503   customerName: this.localSaleCustomerName.trim() || 'Helyszíni vásárló',
2504   customerEmail: this.getLocalSaleEmail(),
2505   customerPhone: this.getLocalSalePhone(),
2506   shipping: {
2507     zip: '-',
2508     city: 'Helyszíni vásárlás',
2509     address: 'Ügyfélszolgálati pult'
2510   },
2511   shippingMethod: {
2512     id: 'local-purchase',
2513     label: 'Helyszíni vásárlás',
2514     fee: 0,
2515     eta: 'Azonnali átvétel'
2516   },
2517   billing: {
2518     sameAsShipping: true,
2519     name: this.localSaleCustomerName.trim() || 'Helyszíni vásárló',
2520     zip: '-',
2521     city: 'Helyszíni vásárlás',
2522     address: 'Ügyfélszolgálati pult'
2523   },
2524   paymentMethod: {
2525     id: this.localSalePaymentMethod,
2526     label: paymentLabel,
2527     fee: 0,
2528     dueDays: this.localSalePaymentMethod === 'transfer' ? this.getLocalSalePaymentDeadlineDays() :
      undefined,
2529     dueAt: paymentDueAt
2530   },
2531   business: {
2532     isBusinessBuyer: this.localSaleIsBusinessBuyer,
2533     companyName: this.localSaleIsBusinessBuyer
      ? (this.localSaleCompanyName.trim() || this.localSaleCustomerName.trim())
      : undefined,
2534     taxNumber: this.localSaleIsBusinessBuyer ? this.localSaleTaxNumber.trim() : undefined
2535   },
2536   salesChannel: 'local-admin',
2537   pricing: {
2538     subtotal,
2539     shippingFee: 0,
2540     paymentFee: 0,
2541     discount,
2542     total
2543   },
2544   couponCode: '',
2545   comment: this.localSaleComment.trim() || this.getLocalSaleDiscountLabel() || ' admin
2546   ',
2547   items,
2548   total,
2549 }
```

M2.10g. ábra: Helyszíni számla és PDF generálás indítása. A részlet azt mutatja, hogyan kéri le a rendszer a számlaazonosítót, majd hogyan indítja el a helyszíni vásárláshoz tartozó PDF bizonylat letöltését.

```
M2.10g - Helyszíni számla és pdf indítás

src/pages/admin/admin.ts | 2551-2557. sor
2551 });
2552
2553 let invoiceError: unknown = null;
2554 try {
2555   const invoiceData = await this.orderService.ensureInvoiceForOrder(orderRef.id);
2556   this.invoiceService.downloadInvoicePdf({
2557     id: orderRef.id,
```

M2.11. ábra: Helyszíni vásárlás tranzakciós mentése. A részlet azt szemlélteti, hogyan menti a rendszer a helyszíni vásárlást, miközben kezeli a készletváltozást és az adatkonzisztenciát.

M2.11 - Helyszíni vásárlás tranzakció

```
src/app/services/order.service.ts | 238-304. sor
238  async createLocalSaleOrder(order: Order & { items: CartItem[] }) {
239    // Helyszíni vásárlás tranzakciójában:
240    // 1) készlet ellenőrzés, 2) készlet csökkentés, 3) rendelés mentés.
241    const cleanOrder = this.removeUndefinedDeep(order);
242    this.assertValidLocalSaleOrder(cleanOrder);
243
244    const orderId = await runTransaction(db, async transaction => {
245      const stockUpdates: Array<{
246        ref: ReturnTypedDoc;
247        nextQty: number;
248        nextStock: string;
249      }> = [];
250
251      for (const item of cleanOrder.items) {
252        if (!item.firestoreId) {
253          throw new Error('missing-product-id:${item.name}');
254        }
255
256        const productRef = doc(db, 'products', item.firestoreId);
257        const productSnap = await transaction.get(productRef);
258
259        if (!productSnap.exists()) {
260          throw new Error('product-not-found:${item.name}');
261        }
262
263        const data = productSnap.data() as { stockQuantity?: number; stock?: string };
264        const currentQty = Math.max(0, Number(data.stockQuantity) || 0);
265
266        if (currentQty < item.quantity) {
267          throw new Error('insufficient-stock:${item.name}');
268        }
269
270        const nextQty = Math.max(0, currentQty - item.quantity);
271        const previousStock = data.stock || 'Készleten';
272        const nextStock = this.resolveStockLabel(nextQty, previousStock);
273
274        stockUpdates.push({
275          ref: productRef,
276          nextQty,
277          nextStock
278        });
279      }
280
281      for (const update of stockUpdates) {
282        transaction.update(update.ref, {
283          stockQuantity: update.nextQty,
284          stock: update.nextStock
285        });
286      }
287
288      const orderRef = doc(this.ordersCollection);
289      transaction.set(orderRef, {
290        ...cleanOrder,
291        createdAt: Date.now()
292      });
293
294      return orderRef.id;
295    });
296
297    return { id: orderId };
298  }
299
300  private assertValidLocalSaleOrder(order: Order & { items: CartItem[] }): void {
301    if (!Array.isArray(order.items) || order.items.length === 0) {
302      throw new Error('missing-order-items');
303    }
304  }
```

M2.12a. ábra: Termék törlés és import előkészítés. A kódrészlet bemutatja az egyedi terméktörlést, valamint az adminisztrációs újratöltéshez használt teljes termékkollekció törlésének előkészítését.

```
M2.12a - Termek torles es import elokeszites

src/app/services/product.service.ts | 59-77. sor
59   }
60
61   deleteProduct(productId: string) {
62     const productRef = doc(db, 'products', productId);
63     return deleteDoc(productRef);
64   }
65
66   async deleteAllProducts(): Promise<number> {
67     // Teljes products kollekció törlése admin "nulláról újratöltés" művelethez.
68     const snapshot = await getDocs(this.productsCollection);
69
70     if (snapshot.empty) {
71       return 0;
72     }
73
74     await Promise.all(snapshot.docs.map(item => deleteDoc(doc(db, 'products', item.id))));
75     return snapshot.size;
76   }
77 }
```

M2.12b. ábra: CSV termékek normalizálása és szűrése. A részlet azt mutatja, hogyan alakítja át a rendszer az importált termékadatokat egységes formára, majd hogyan hagyja ki a hiányos rekordokat.

```
M2.12b - CSV termekek normalizalasa

src/app/services/product.service.ts | 78-94. sor
78   async importProductsBulk(
79     products: Product[],
80     mode: 'insert' | 'upsertBySku' = 'upsertBySku'
81   ): Promise<{ created: number; updated: number; skipped: number }> {
82     // CSV importhoz csoportos mentes: új termékek vagy SKU szerinti frissítés.
83     const normalizedProducts = products
84       .map(product => this.normalizeProductPayload(product))
85       .filter(product =>
86         !!product.name &&
87         typeof product.price === 'number' &&
88         !!product.category &&
89         !!product.image
90       );
91
92     if (normalizedProducts.length === 0) {
93       return { created: 0, updated: 0, skipped: products.length };
94     }
95 }
```

M2.12c. ábra: CSV insert módú termékmentés. A kódrészlet bemutatja, hogyan hoz létre a rendszer új termékdokumentumokat, amikor az importálás egyszerű beszúrási módban fut.

```
M2.12c - CSV insert mod termekmentes

src/app/services/product.service.ts | 96-111. sor
96   if (mode === 'insert') {
97     await Promise.all(
98       normalizedProducts.map(product =>
99         addDoc(this.productsCollection, {
100           ...product,
101           createdAt: Date.now()
102         })
103     );
104   };
105
106   return {
107     created: normalizedProducts.length,
108     updated: 0,
109     skipped: Math.max(0, products.length - normalizedProducts.length)
110   };
111 }
```

M2.12d. ábra: CSV upsert SKU alapú frissítés. A részlet bemutatja, hogyan párosítja a rendszer a CSV-ben érkező termékeket a meglévő Firestore dokumentumokkal cikkszám alapján, majd hogyan frissít vagy hoz létre terméket batch műveletekkel.

M2.12d - CSV upsert SKU alapu frissites

```
src/app/services/product.service.ts | 113-166. sor
113   const existingSnapshot = await getDocs(this.productsCollection);
114   const skuToDocId = new Map<string, string>();
115
116   for (const item of existingSnapshot.docs) {
117     const sku = ((item.data() as Partial<Product>).sku || '').toString().trim().toLowerCase();
118     if (sku) {
119       skuToDocId.set(sku, item.id);
120     }
121   }
122
123   let created = 0;
124   let updated = 0;
125   let skipped = Math.max(0, products.length - normalizedProducts.length);
126   let batch = writeBatch(db);
127   let opCount = 0;
128
129   const commitBatch = async (force = false) => {
130     // Firestore batch limit 500 művelet, ezért 450-nél biztonsági ráhagyással commitolunk.
131     if (opCount >= 450 || (force && opCount > 0)) {
132       await batch.commit();
133       batch = writeBatch(db);
134       opCount = 0;
135     }
136   };
137
138   for (const product of normalizedProducts) {
139     const skuKey = (product.sku || '').toString().trim().toLowerCase();
140
141     if (!skuKey) {
142       skipped += 1;
143       continue;
144     }
145
146     const existingId = skuToDocId.get(skuKey);
147     if (existingId) {
148       const productRef = doc(db, 'products', existingId);
149       batch.update(productRef, {
150         ...product,
151         updatedAt: Date.now()
152       });
153       updated += 1;
154     } else {
155       const productRef = doc(this.productsCollection);
156       batch.set(productRef, {
157         ...product,
158         createdAt: Date.now()
159       });
160       created += 1;
161       skuToDocId.set(skuKey, productRef.id);
162     }
163
164     opCount += 1;
165     await commitBatch();
166   }
```

M2.13a. ábra: Számlaszám generálása tranzakcióban. A kódrészlet bemutatja, hogyan használ a rendszer éves számlaszámlálót, hogyan kerüli el a duplikált számlaszámot, és hogyan menti vissza a számlaadatokat a rendeléshez.

M2.13a - Számlaszám generálás tranzakcióban

```
src/app/services/order.service.ts | 327-362. sor
327   async ensureInvoiceForOrder(orderId: string): Promise<{ invoiceNumber: string; invoicedAt: number }> {
328     // Számlaszám generálás évi futószámmal (INV-YYYY-XXXX), tranzakción.
329     const orderRef = doc(db, 'orders', orderId);
330     const year = new Date().getFullYear().toString();
331     const counterRef = doc(db, 'invoiceCounters', year);
332
333     return runTransaction(db, async transaction => {
334       const orderSnap = await transaction.get(orderRef);
335       if (!orderSnap.exists()) {
336         throw new Error('order-not-found');
337       }
338
339       const orderData = orderSnap.data() as Order;
340       if (orderData.invoiceNumber && orderData.invoicedAt) {
341         return {
342           invoiceNumber: orderData.invoiceNumber,
343           invoicedAt: orderData.invoicedAt
344         };
345       }
346
347       const counterSnap = await transaction.get(counterRef);
348       const currentSeq = counterSnap.exists()
349         ? Math.max(0, Number((counterSnap.data() as { seq?: number }).seq) || 0)
350         : 0;
351       const nextSeq = currentSeq + 1;
352       const padded = String(nextSeq).padStart(4, '0');
353       const invoiceNumber = `INV-${year}-${padded}`;
354       const invoicedAt = Date.now();
355
356       transaction.set(counterRef, { seq: nextSeq, updatedAt: invoicedAt }, { merge: true });
357       transaction.update(orderRef, { invoiceNumber, invoicedAt });
358
359       return { invoiceNumber, invoicedAt };
360     });
361   }
362 }
```

M2.13b. ábra: PDF bizonylat adatainak előkészítése. A részlet azt mutatja, hogyan állítja össze a rendszer a számlán megjelenő vevői, szállítási, fizetési, tétel- és összegadatokat.

M2.13b - PDF bizonylat adatainak előkészítése

```
src/app/services/invoice.service.ts | 8-40. sor
8   export class InvoiceService {
9     downloadInvoicePdf(order: Order & { id: string }): void {
10      // Rendelés adat -> PDF byte tömb -> bingesztes letöltés.
11      const issuer = environment.invoiceIssuer;
12      const invoiceNumber = order.invoiceNumber || `INV-${order.id}`;
13      const issueDate = order.invoicedAt || Date.now();
14      const paymentLabel = order.paymentMethod?.label || '-';
15      const shippingLabel = order.shippingMethod?.label || '-';
16      const buyerAddress = order.salesChannel === 'local-admin'
17        ? `${order.shipping?.city || 'Helyszíni vásárlás'}, ${order.shipping?.address || 'Ügyfeliszolgalati pult'}`
18        : `${order.shipping?.zip || '-'}, ${order.shipping?.city || '-'}, ${order.shipping?.address || '-'}`;
19      const buyerCompany = order.business?.isBusinessBuyer
20        ? (order.business?.companyName || order.customerName)
21        : '-';
22      const buyerTaxNumber = order.business?.isBusinessBuyer ? (order.business?.taxNumber || '-') : '-';
23      const netTotal = Math.round(order.total / 1.27);
24      const vatTotal = Math.max(0, order.total - netTotal);
25      const pdfBytes = this.buildInvoicePdf({
26        issuerName: issuer.name,
27        issuerAddress: issuer.address,
28        issuerTaxNumber: issuer.taxNumber,
29        invoiceNumber,
30        issueDate: new Date(issueDate).toLocaleDateString('hu-HU'),
31        orderId: order.id,
32        salesChannel: order.salesChannel === 'local-admin' ? 'Helyszíni' : 'Webes',
33        buyerName: order.customerName,
34        buyerEmail: order.customerEmail,
35        buyerPhone: order.customerPhone,
36        buyerCompany,
37        buyerTaxNumber,
38        buyerAddress,
39        paymentLabel,
40        shippingLabel,
```

M2.13c. ábra: PDF letöltés indítása a böngészőben. A kódrészlet bemutatja, hogyan készül a PDF byte tömbből Blob, majd hogyan jön létre az ideiglenes letöltési hivatkozás a számla fájlnevével történő mentéséhez.

M2.13c - PDF letöltes bongeszobon

```
src/app/services/invoice.service.ts | 41-56. sor
41     items: order.items.map(item => ({
42       name: item.name,
43       quantity: item.quantity,
44       unitPrice: item.price,
45       gross: item.price * item.quantity
46     })),
47     netTotal,
48     vatTotal,
49     grossTotal: order.total
50   });
51   const arrayBuffer = pdfBytes.buffer.slice(pdfBytes.byteOffset, pdfBytes.byteOffset + pdfBytes.byteLength) as
    ArrayBuffer;
52   const blob = new Blob([arrayBuffer], { type: 'application/pdf' });
53   const url = URL.createObjectURL(blob);
54   const link = document.createElement('a');
55   const stamp = new Date(issueDate).toISOString().replace(/[:.]/g, '-');
56   link.href = url;
```

M2.14a. ábra: AI kérdés domain szűrése. A kódrészlet bemutatja, hogyan engedi tovább a rendszer csak a webshophoz és épületgépészeti témákhoz kapcsolódó kérdéseket, illetve hogyan ad visszajelzést irreleváns kérdés esetén.

M2.14a - AI keres domain szurese

```
src/app/services/chatbot-llm.service.ts | 39-59. sor
39   async recommend(userMessage: string, products: Product[]): Promise<LLMRecommendationResult | null> {
40     const endpoint = environment.aiAssistantEndpoint;
41     if (!endpoint) {
42       return null;
43     }
44
45     if (!this.isDomainQuestion(userMessage)) {
46       return {
47         text: 'Ebben csak a TDL Webshop termekeivel, rendeleseivel es epuletgepeszeti temakkal kapcsolatban
          tudok segiteni. Irj peldaul futesrol, vizszerelesrol, klimarol, szellozesrol vagy egy konkret
          termekrol.',
48         products: []
49       };
50     }
51
52     const relevantCatalog = this.buildRelevantCatalog(userMessage, products);
53     const noExactCatalogMatchText = 'Ehhez a kereshez most nem talaltam eleg pontosan illeszkedo termeket a
          katalogusban. Altalanos epuletgepeszeti iranyt tudok adni, de pontos ajanlatert es beszerezhetosegert
          erdemes emailben vagy személyesen felkeresni az uzletet.';
54     if (relevantCatalog.length === 0) {
55       return {
56         text: noExactCatalogMatchText,
57         products: []
58       };
59     }
59   }
```

M2.14b. ábra: AI proxy hívás és válaszfeldolgozás. A részlet azt mutatja, hogyan küldi el a kliensoldal a felhasználói üzenetet és a releváns termékkatalógust a proxy felé, majd hogyan dolgozza fel a válaszban kapott szöveget és termékJánlásokat.

M2.14b - AI proxy hívás és válaszfeldolgozás

```
src/app/services/chatbot-llm.service.ts | 61-88. sor
61 const response = await fetch(endpoint, {
62   method: 'POST',
63   headers: {
64     'Content-Type': 'application/json'
65   },
66   body: JSON.stringify({
67     message: userMessage,
68     products: relevantCatalog
69   })
70 });
71
72 if (!response.ok) {
73   throw new Error('AI proxy hiba (${response.status})');
74 }
75
76 const data = await response.json();
77 const text = (data?.text || '').toString().trim();
78 const names = Array.isArray(data?.productNames) ? data.productNames : [];
79 const skus = Array.isArray(data?.productSkus) ? data.productSkus : [];
80 const selected = this.pickProducts(names, skus, products, relevantCatalog);
81 const proxySuggestedProducts = names.length > 0 || skus.length > 0;
82 const safeText = proxySuggestedProducts && selected.length === 0
83   ? noExactCatalogMatchText
84   : this.ensureContactNote(text || this.buildCatalogFallbackText(relevantCatalog), selected.length > 0);
85
86 return {
87   text: safeText,
88   products: selected
89 }
```

M2.14c. ábra: AI releváns termékkatalógus összeállítása. A kódrészlet bemutatja, hogyan normalizálja a rendszer a felhasználói üzenetet, hogyan szűri a készleten lévő és kategóriában illeszkedő termékeket, majd hogyan pontozza és rendezi az ajánlható termékeket.

M2.14c - AI releváns termékkatalógus

```
src/app/services/chatbot-llm.service.ts | 92-114. sor
92 private buildRelevantCatalog(userMessage: string, products: Product[]): AssistantCatalogProduct[] {
93   const normalizedMessage = this.normalize(userMessage);
94   const tokens = normalizedMessage.split(/\s+/).filter(token => token.length > 2);
95   const allowedCategories = this.detectAllowedCategories(normalizedMessage);
96   const requiredProductTerms = this.detectRequiredProductTerms(normalizedMessage);
97
98   const scored = products
99     .filter(product => Number(product.stockQuantity || 0) > 0)
100     .filter(product => this.isAllowedCategory(product, allowedCategories))
101     .filter(product => this.matchesRequiredProductIntent(product, requiredProductTerms))
102     .map(product => ({
103       product,
104       score: this.scoreProduct(product, normalizedMessage, tokens)
105     })))
106     .sort((left, right) => right.score - left.score);
107
108   const relevant = scored.filter(item => item.score >=
109     this.minimumCatalogScore(requiredProductTerms)).slice(0, 28);
110   if (relevant.length > 0) {
111     return relevant.map(({ product }) => this.toCatalogProduct(product));
112   }
113   return [];
114 }
```

M2.14d. ábra: AI katalógus termék DTO létrehozása. A részlet bemutatja, hogyan alakítja át a rendszer a belső termékmodellt az AI-asszisztens számára küldhető, csak szükséges mezőket tartalmazó katalógusobjektummá.

M2.14d - AI katalogus termék DTO

```
src/app/services/chatbot-llm.service.ts | 116-128. sor
116   private toCatalogProduct(product: Product): AssistantCatalogProduct {
117     return {
118       id: product.id || '',
119       name: product.name,
120       sku: product.sku || '',
121       brand: product.brand || '',
122       category: product.category,
123       price: Number(product.price || 0),
124       stockQuantity: Number(product.stockQuantity || 0),
125       shortDescription: product.shortDescription || '',
126       description: product.description || ''
127     };
128   }
```

M2.15a. ábra: Worker CORS és metódusvédelem. A kódrészlet bemutatja, hogyan kezeli a proxy az OPTIONS preflight kéréseket, és hogyan engedélyezi csak a POST metódust az AI-asszisztens használatához.

M2.15a - Worker CORS es metodus vedelem

```
workers/openrouter-proxy/src/index.js | 153-160. sor
153   async fetch(request, env) {
154     if (request.method === 'OPTIONS') {
155       return new Response('', { status: 204, headers: corsHeaders(request) });
156     }
157
158     if (request.method !== 'POST') {
159       return jsonResponse(request, { error: 'method-not-allowed' }, 405);
160     }
161   }
```

M2.15b. ábra: Worker rate limit és API-kulcs ellenőrzés. A részlet azt mutatja, hogyan korlátozza a rendszer a túl gyakori AI-kéréseket, és hogyan akadályozza meg a működést hiányzó OpenRouter kulcs esetén.

M2.15b - Worker rate limit es kulcsellenorzes

```
workers/openrouter-proxy/src/index.js | 162-180. sor
162   if (isRateLimited(request, env)) {
163     return jsonResponse(request, {
164       error: 'rate-limited',
165       text: 'Túl sok AI keres érkezett rövid időn belül. Kérlek próbald újra később.',
166       productNames: [],
167       productSkus: []
168     }, 429);
169   }
170
171   if (!env.OPENROUTER_API_KEY) {
172     return jsonResponse(request, { error: 'assistant-not-configured' }, 500);
173   }
174
175   let body;
176   try {
177     body = await request.json();
178   } catch {
179     return jsonResponse(request, { error: 'invalid-json' }, 400);
180   }
```

M2.15c. ábra: Worker üzenet- és domainvalidáció. A kódrészlet bemutatja, hogyan olvassa ki a proxy a felhasználói üzenetet és a termékkatalógust, valamint hogyan szűri ki a webshop témakörén kívüli kérdéseket.

```
M2.15c - Worker üzenet és domain validáció

workers/openrouter-proxy/src/index.js | 182-197. sor
182   const message = String(body?.message || '').trim();
183   const requestedProducts = Array.isArray(body?.products) ? body.products : [];
184
185   if (!message) {
186     return jsonResponse(request, { error: 'missing-message' }, 400);
187   }
188
189   if (!isDomainQuestion(message)) {
190     return jsonResponse(request, {
191       text: 'Ebben csak a TDL Webshop termékeivel, rendeléseivel és epületgepeszeti temakkal kapcsolatban tudok segíteni.',
192       productNames: [],
193       productSkus: []
194     });
195   }
196
197   const catalog = sanitizeCatalog(requestedProducts);
```

M2.15d. ábra: OpenRouter API hívás. A részlet azt szemlélteti, hogyan hívja meg a Cloudflare Worker az OpenRouter chat completions végpontját, miközben a titkos API-kulcs a szerveroldali környezetben marad.

```
M2.15d - OpenRouter API hívás

workers/openrouter-proxy/src/index.js | 198-216. sor
198   const openRouterResponse = await fetch('https://openrouter.ai/api/v1/chat/completions', {
199     method: 'POST',
200     headers: {
201       'Content-Type': 'application/json',
202       Authorization: `Bearer ${env.OPENROUTER_API_KEY}`,
203       'HTTP-Referer': env.SITE_URL || 'https://tdlwebshop.web.app',
204       'X-Title': 'TDL Webshop AI Assistant'
205     },
206     body: JSON.stringify({
207       model: env.OPENROUTER_MODEL || 'openrouter/auto',
208       temperature: 0.15,
209       max_tokens: 650,
210       messages: [
211         { role: 'system', content: buildSystemPrompt() },
212         { role: 'user', content: buildUserPrompt(message, catalog) }
213       ]
214     })
215   });
216
```

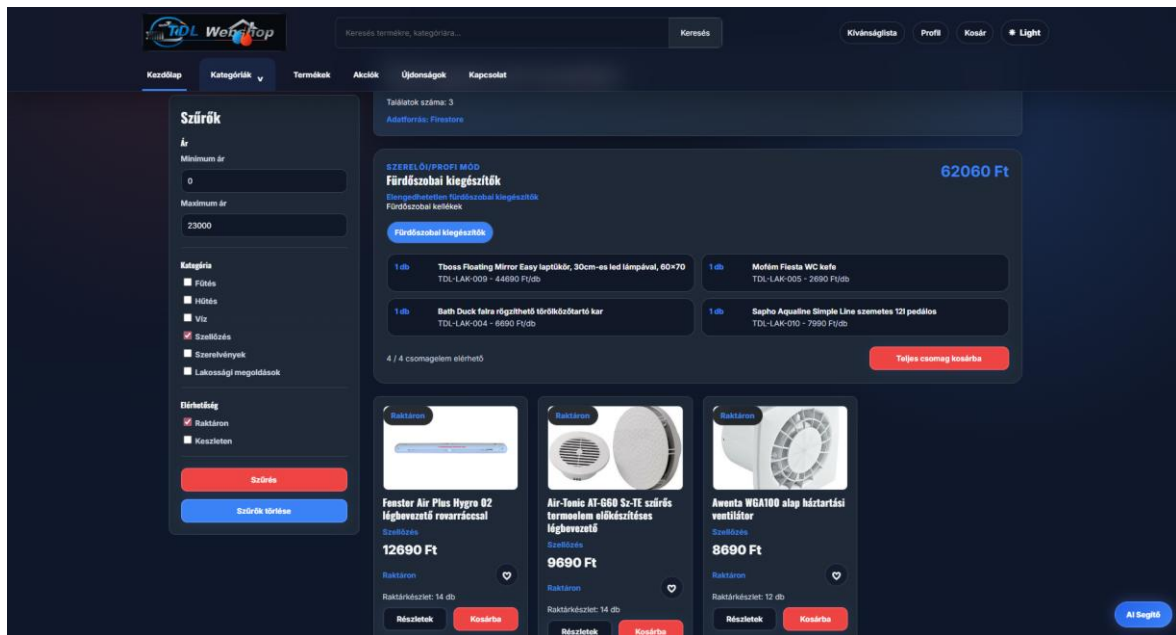
M2.15e. ábra: OpenRouter válasz feldolgozása. A kódrészlet bemutatja, hogyan ellenőrzi a proxy a külső AI-szolgáltatás válaszát, majd hogyan alakítja át a modell által adott szöveget a frontend számára értelmezhető eredménnyé.

```
M2.15e - OpenRouter válasz parse

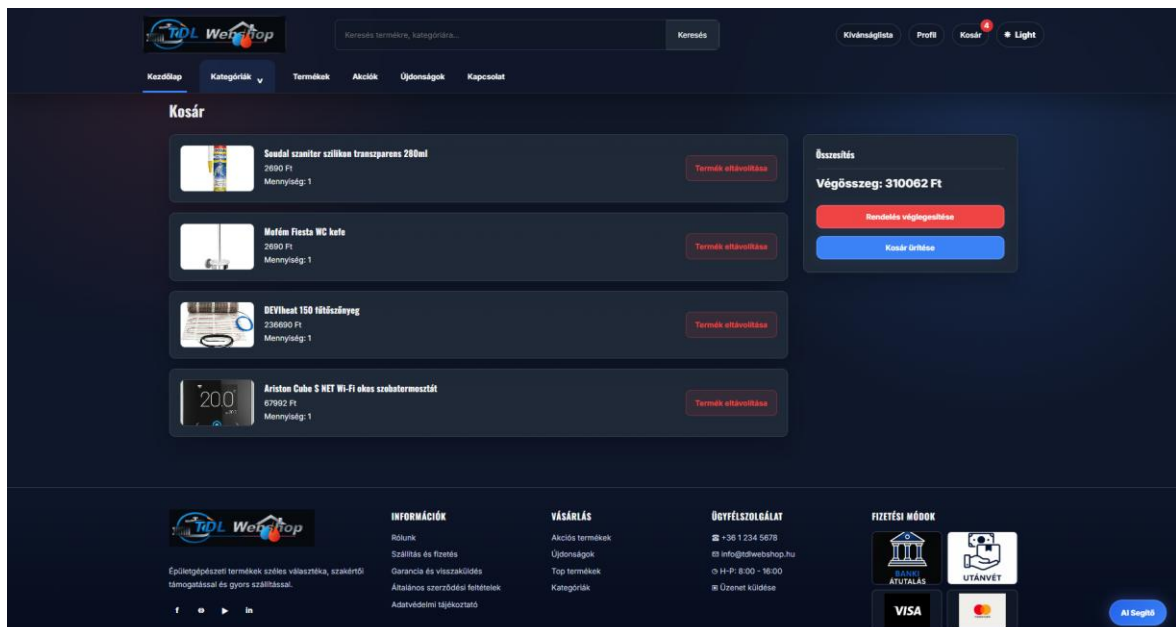
workers/openrouter-proxy/src/index.js | 217-224. sor
217   if (!openRouterResponse.ok) {
218     return jsonResponse(request, { error: 'assistant-upstream-error' }, 502);
219   }
220
221   const aiData = await openRouterResponse.json();
222   const outputText = aiData?.choices?.[0]?.message?.content || '';
223   const parsed = parseJsonResult(outputText);
224
```

M3. Képernyőképek teljes gyűjteménye

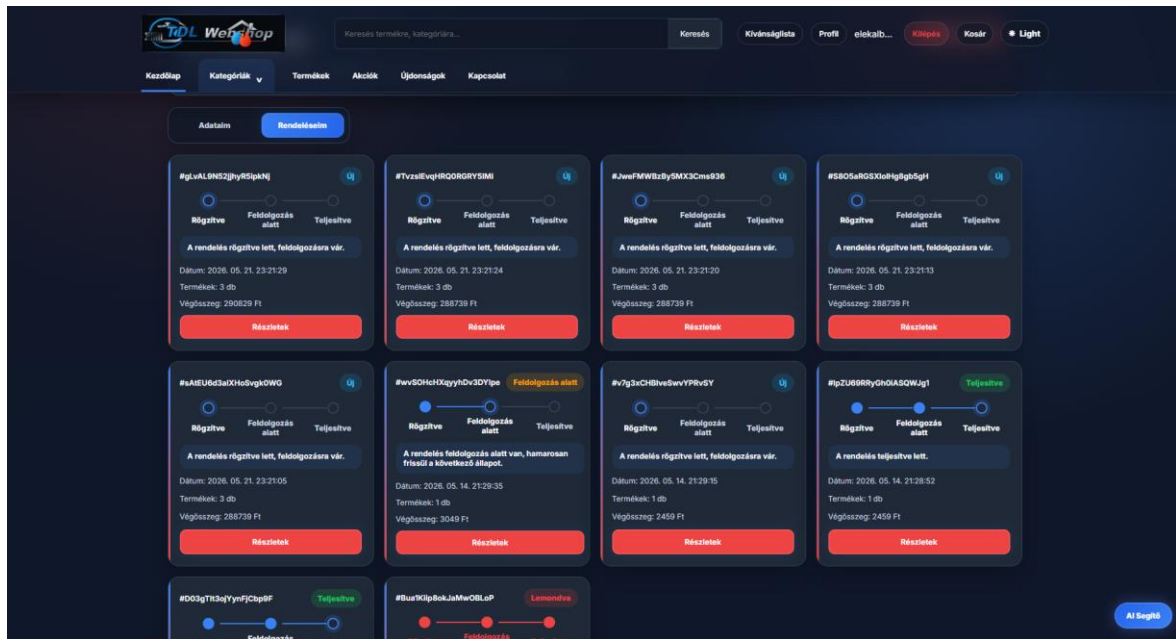
A főszövegben a legjellemzőbb felhasználói folyamatok képernyőképei szerepelnek (1–9. ábra). A részletes képgyűjteményt – beleértve a termékadatlapot, a kívánságlistát, az admin termékkezelést, a CSV-importot, a készletfigyelést, a felhasználó- és jogosultságkezelést, valamint a több állapotú nézeteket (loading, empty, error, success) – ez a melléklet, illetve a repó docs/ux/screenshots/ mappája tartalmazza. A képek a docs/ux/screens.csv leíró táblával együtt értelmezhetők.



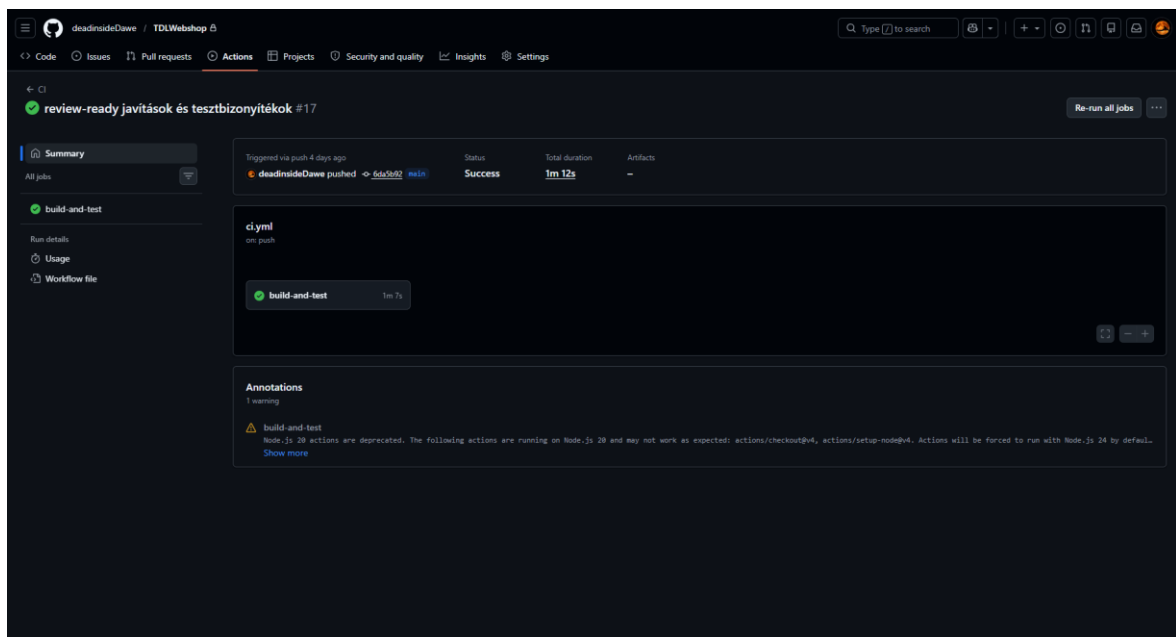
M3.1. ábra. Terméklista szűrési és keresési lehetőségekkel.



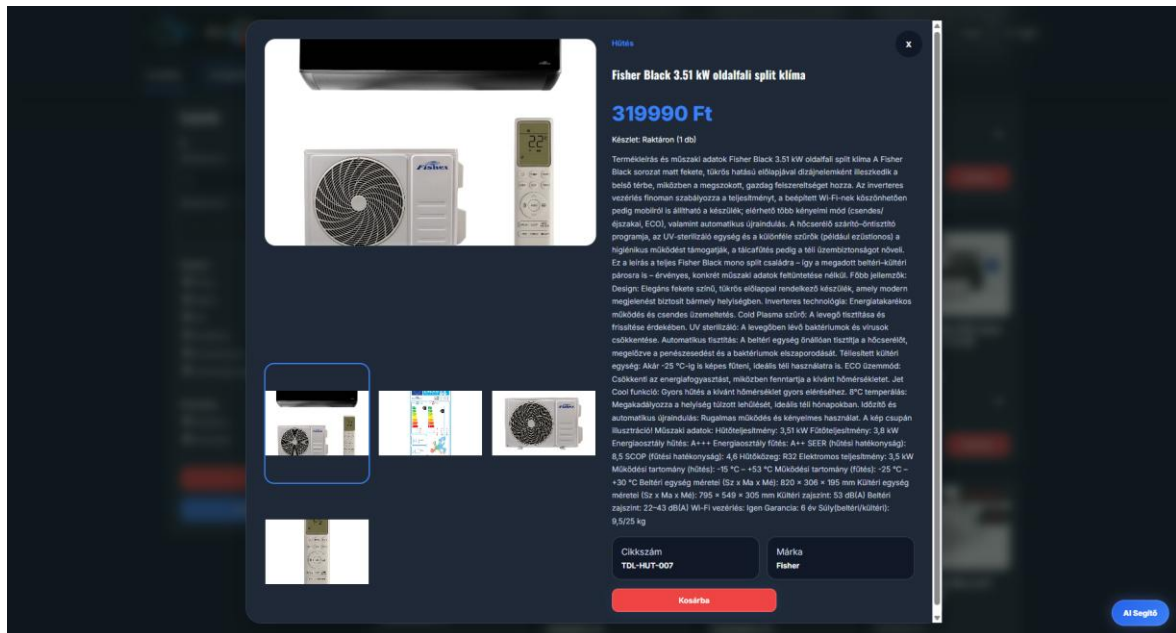
M3.2. ábra. Kosár több termékkel, összegzéssel és tételenkénti törléssel.



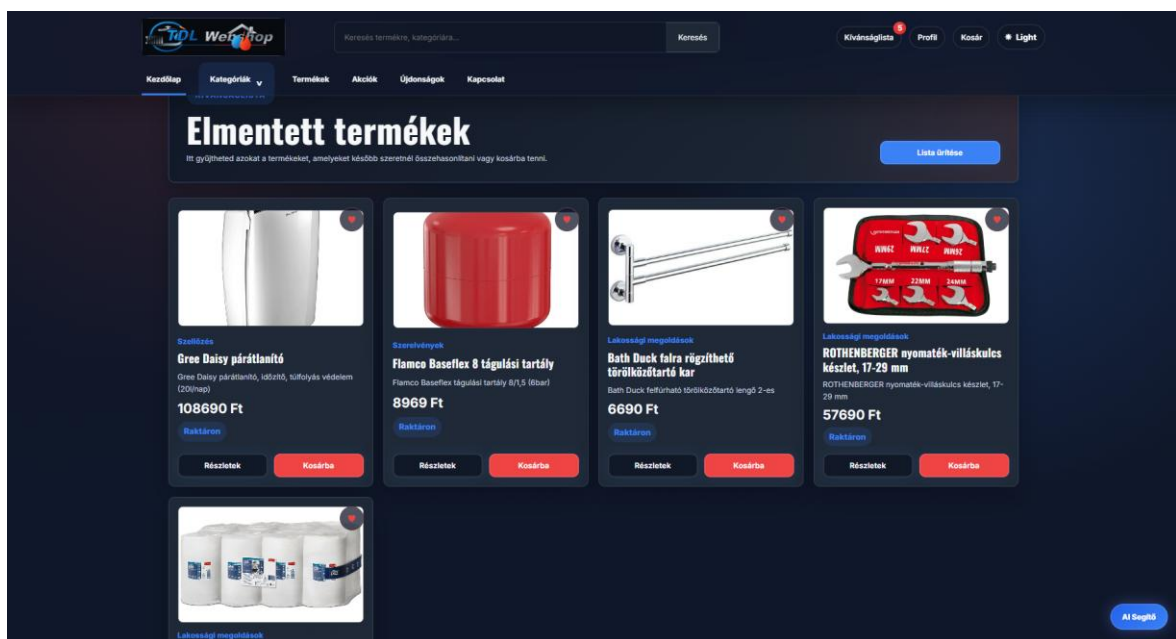
M3.3. ábra. Vásárlói profil rendeléstörténettel és státuszokkal.



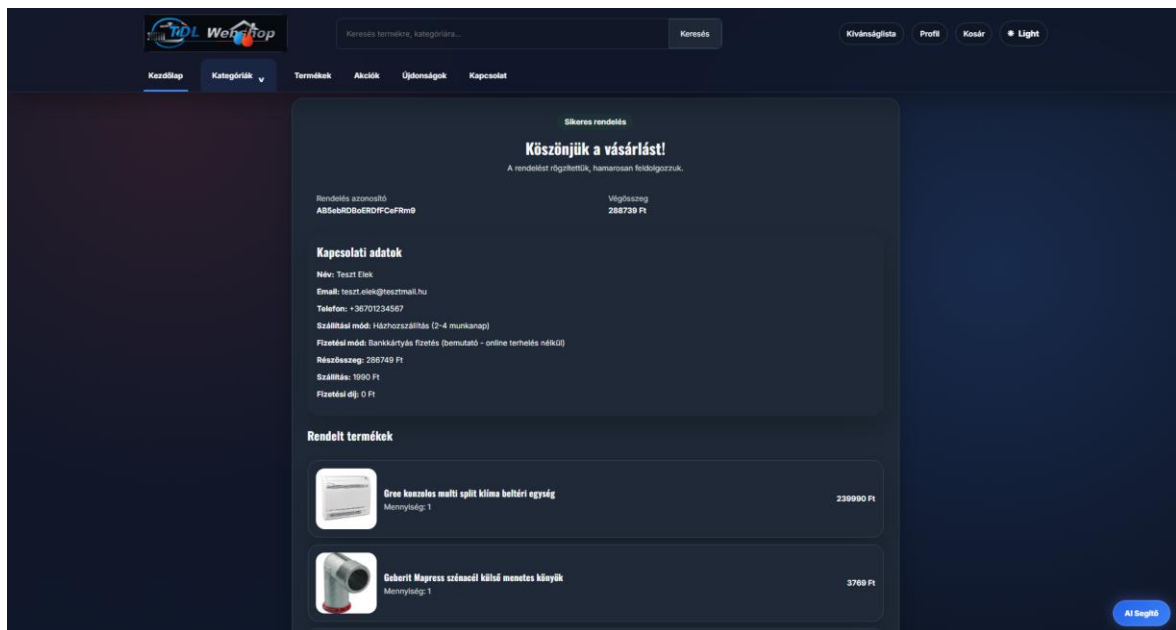
M3.4. ábra. GitHub Actions – zöld CI-futás (build és tesztek).



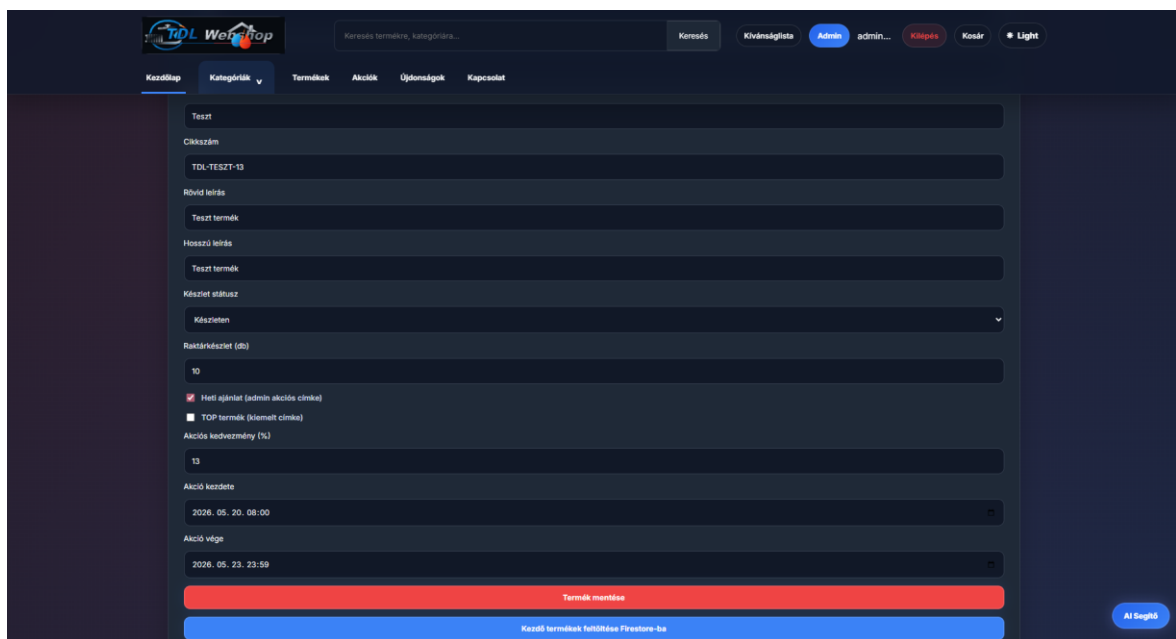
M3.5. ábra. Termékadatlap részletes nézetrel.



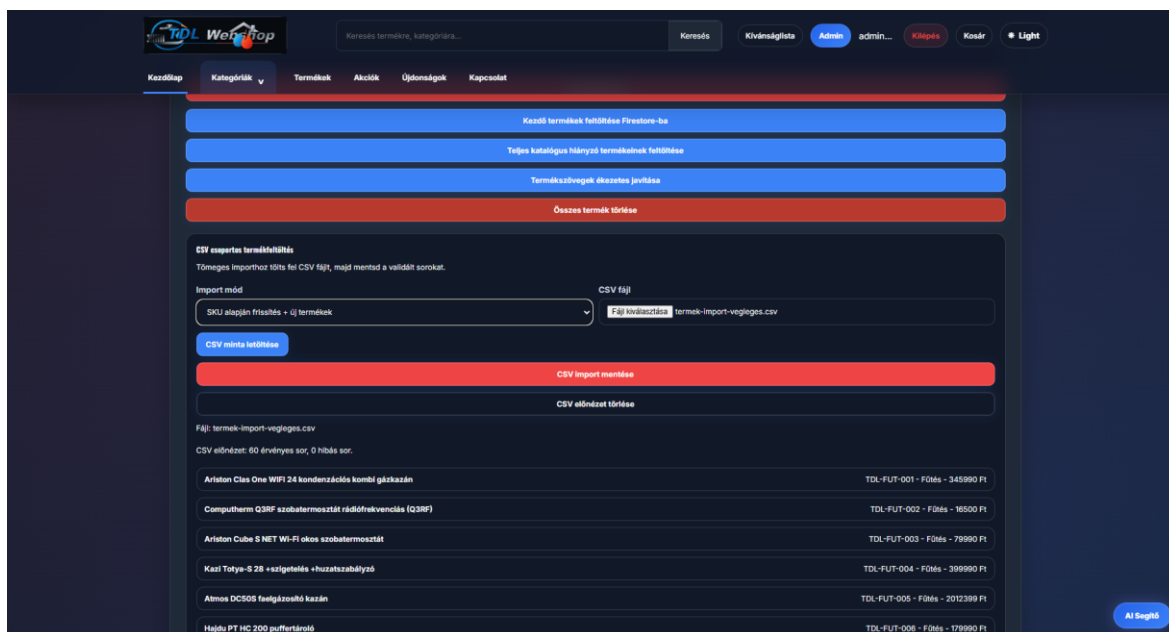
M3.6. ábra. Kívánságlista – elmentett termékek.



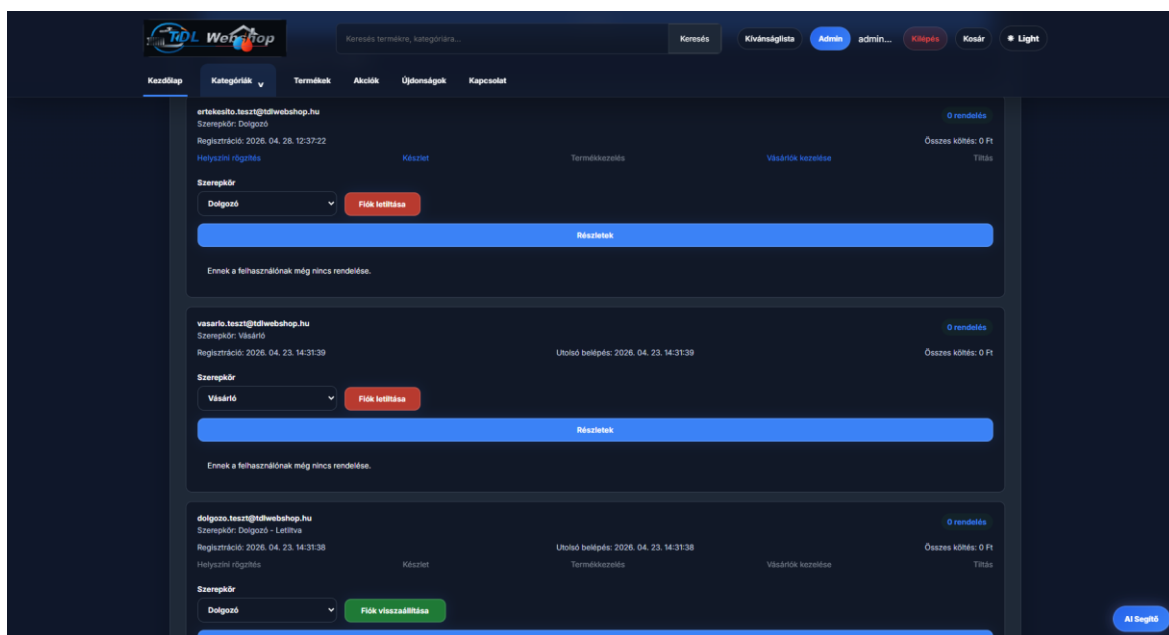
M3.7. ábra. Sikeres rendelés visszajelzés.



M3.8. ábra. Admin termékkezelés / új termék felvitele.



M3.9. ábra. CSV-import felület.



M3.10. ábra. Felhasználó- és jogosultságkezelés.

M4. Elektronikus melléklet és reprodukciós lépések

A szakdolgozat elektronikus mellékletei az alábbi helyeken érhetők el. A teljes forráskódot tartalmazó GitHub-repó: <https://github.com/deadinsideDawe/TDLWebshop> (végleges branch: main, végleges commit hash: `f3c88ff1f1ec8b0d918ba5f52a37c2a1e2dfd60c`). A leadott állapot Coospace-zip formátumban: TothDavid_TDLWebshop_szakdolgozat.zip. A futtatható deploy elérhetősége: <https://tdlwebshop.web.app>. A reprodukciós lépéseket a 12. fejezet és a repó README.md fájlja írja le; a futtatáshoz szükséges környezeti változók mintáját a repó .env.example fájlja tartalmazza.

Nyilatkozat

Alulírott Tóth Dávid László gazdaságinformatikus BSc szakos hallgató kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem Informatikai Intézetében készítettem, a gazdaságinformatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, az saját munkám eredménye, és csak a hivatkozott forrásokat, szakirodalmat és eszközöket használtam fel. Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Szeged, 2026.05.22.

Tóth Dávid László